

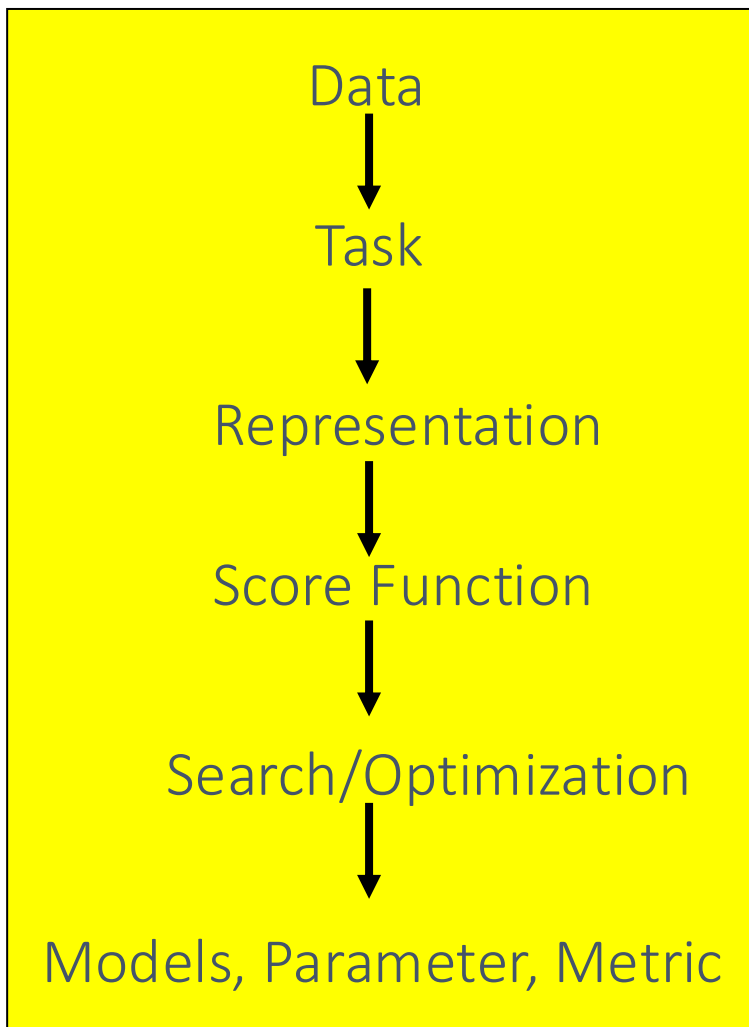
UVA CS 4774: Machine Learning

Lecture 11: Logistic Regression

Dr. Yanjun Qi

University of Virginia
Department of Computer Science

Machine Learning in a Nutshell



ML grew out of
work in AI

Optimize a
performance criterion
using example data or
past experience,

Aiming to generalize to
unseen data

Past Course Content Plan → Regarding Tasks

JTD

- ☒ ~~Regression (supervised)~~
- ☒ ~~Learning theory~~
- ☐ Classification (supervised)
- ☐ Unsupervised models
- ☐ Deep Learning models
- ☐ Reinforcement Learning

Y is a continuous

About $f()$

Y is a discrete

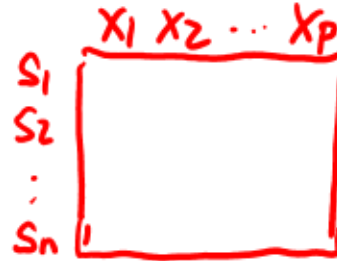
NO Y

About interactions among $Y, X_1, \dots, X_p$

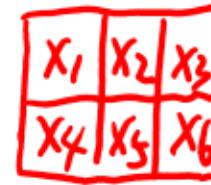
Learn to Interact with environment

Different Type of Data

- Tabular / Matrix



- 2D Grid Structured: Imaging



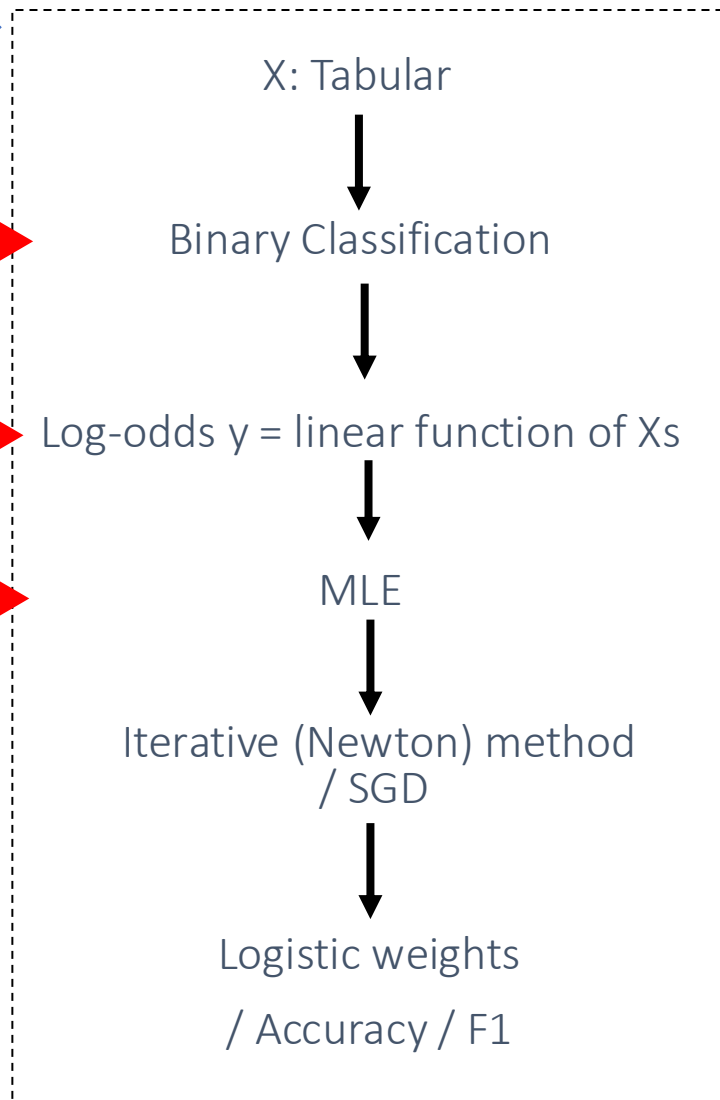
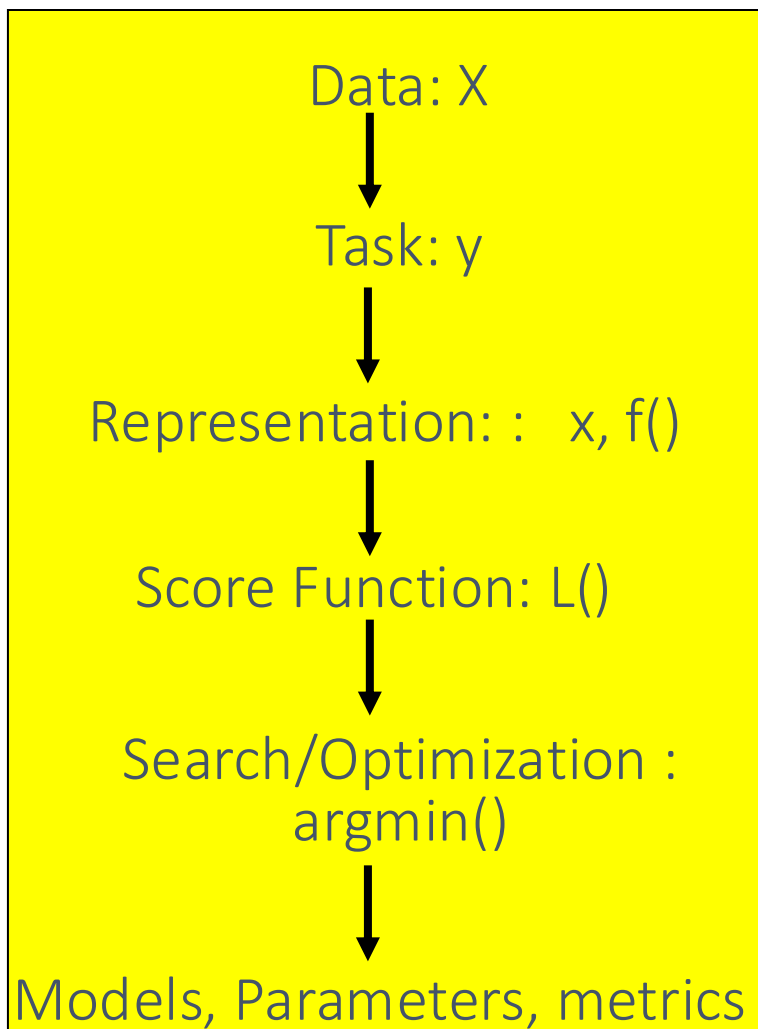
- 1D Sequential Structured: Text

- Graph Structured (Relational)

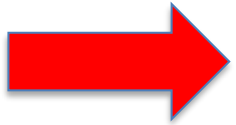
- Set Structured / 3D / Robotic Action Data

Today: Logistic Regression **Classifier**

$$P(y = 1|x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$



Roadmap



- ☐ Bayes Classifier: Classification MAP Rule
- ☐ Logistic Regression Representation
- ☐ Training LG by MLE

Bayes classifiers

- Treat each feature attribute and the class label as random variables.

$$C: \{c_1, \dots, c_L\}$$

Bayes classifiers

$\{C_1, \dots, C_L\}$

- Treat each feature attribute and the class label as random variables.

- **Training**: What is an appropriate loss for classification?

- E.g. Compare training class to output class: Zero-One loss (per class)

$$L(\hat{y}, y) = I(\hat{y} \neq y),$$

$\hat{y}$: $\begin{cases} C_1 \\ C_2 \\ \vdots \\ C_L \end{cases}$

y_{true}

	C_1	C_2	$\dots$	C_L
C_1	0			1
C_2		0		
$\vdots$			$\ddots$	
C_L	1			0

- Can we model/estimate $p(Y=C_i | x) = p(C_i | x_1, x_2, \dots, x_p)$ directly from reference data?

Bayes classifiers

- Treat each feature attribute and the class label as random variables.
- **Testing**: Given a sample vector $\mathbf{x}$ with attributes as $(x_1, x_2, \dots, x_p)$:
 - Goal is to predict its class
 - HOW: $\rightarrow$ we want to find the class that maximizes $p(c | x_1, x_2, \dots, x_p)$.

A handwritten red diagram illustrating the goal of finding the class that maximizes the posterior probability. A red arrow points from the phrase "find the class that maximizes" in the text above to a list of posterior probabilities: $p(c_1 | \vec{x})$, $p(c_2 | \vec{x})$, a vertical ellipsis, and $p(c_L | \vec{x})$. A large red curly bracket groups these probabilities, with an arrow pointing from the bracket to the expression $\max \rightarrow c^*$.

$$\left. \begin{array}{l} p(c_1 | \vec{x}) \\ p(c_2 | \vec{x}) \\ \vdots \\ p(c_L | \vec{x}) \end{array} \right\} \max \rightarrow c^*$$

Bayes Classifiers – Predict via MAP Rule

Task: Classify a new instance X : $X = \langle X_1, X_2, \dots, X_p \rangle$
based on:

$$c_{MAP} = \operatorname{argmax}_{c_j \in C} P(c_j \mid x_1, x_2, \dots, x_p)$$



MAP Rule

MAP = Maximum A posteriori Probability

Bayes Classifiers – Predict via MAP Rule

- Establishing a probabilistic model for classification

→ MAP classification rule

→ To Assign x to c^* if

$$P(C = c^* | \mathbf{X} = \mathbf{x}) \geq P(C = c | \mathbf{X} = \mathbf{x})$$

$$\text{for } c^* \in \mathcal{C}, \mathcal{C} = \{c_1, \dots, c_L\}$$

$$\sum_{j=1}^L P(C=c_j|x)=1$$

Recap: Statistical Decision Theory (Extra)

- Random input vector: X
- Random output variable: Y
- Joint distribution: $\Pr(X, Y)$
- Loss function $L(Y, f(X))$
- Expected prediction error (EPE):

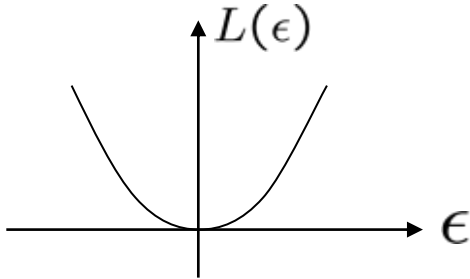
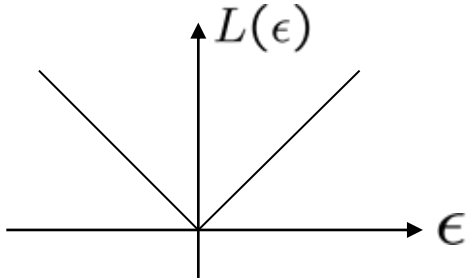
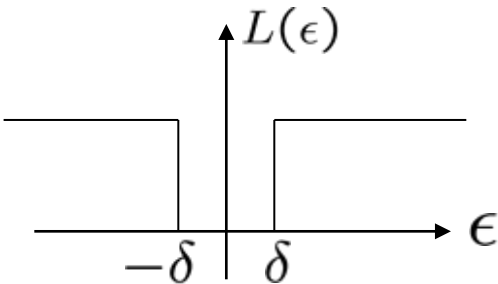
$$\text{EPE}(f) = E(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

$$\text{e.g.} = \int \underbrace{(y - f(x))^2}_{\text{e.g. Squared error loss (also called L2 loss) for regression}} \Pr(dx, dy)$$

e.g. Squared error loss (also called L2 loss) for regression

Consider
population
distribution

SUMMARY: WHEN Expected prediction error (EPE) USES DIFFERENT LOSS (Extra)

Loss Function	Estimator $\hat{f}(x)$
L_2 	$\hat{f}(x) = E[Y X = x]$ <i>regression / KNN</i>
L_1 	$\hat{f}(x) = \text{median}(Y X = x)$
0-1 	$\hat{f}(x) = \arg \max_Y P(Y X = x)$ (Bayes classifier / MAP)

$$f : X \rightarrow C$$

Output is one of the
Discrete Class Label

$C_1, C_2, \dots, C_L$

Establishing a probabilistic model
for classification

$$c_{MAP} = \operatorname{argmax}_{c_j \in C} P(c_j | x_1, x_2, \dots, x_p)$$

$$P(c | \tilde{x})$$

Two Ways MAP:

Generative

$$\operatorname{argmax}_{c \in C} P(c | X) = \operatorname{argmax}_{c \in C} \frac{P(X, c)}{P(X)} = \operatorname{argmax}_{c \in C} P(X | c) P(c)$$

Later!

Discriminative

$$\operatorname{argmax}_{c \in C} P(c | X) = \{c_1, \dots, c_L\}$$

Three major groups of classifiers

- We can divide the large variety of classification approaches into roughly three major types

1. Discriminative

directly estimate a decision rule/boundary

e.g., support vector machine, decision tree, logistic regression,

e.g. neural networks (NN), deep NN

$$P(c|\vec{x})$$

~~2. Generative:~~

~~build a generative statistical model~~

~~e.g., Bayesian networks, Naïve Bayes classifier~~

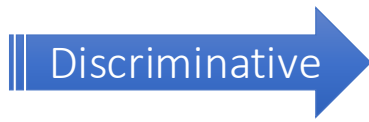
$$P(c|\vec{x}) = \text{Bayes Rule}$$

3. Instance based classifiers

- Use observation directly (no models)

~~e.g. K nearest neighbors~~

Section 2:



$X \rightarrow C$

$$: \underbrace{p(C|X)}_{f(X)}$$

Section 2:



$$: \underbrace{P(c|x)}_{f(x)}$$

– Discriminative Classifiers

$$\arg \max_{c \in C} P(c | \mathbf{X}), C = \{c_1, \dots, c_L\}$$

$$P(c_1 | \mathbf{x}) \quad P(c_2 | \mathbf{x}) \quad \dots \quad P(c_L | \mathbf{x})$$

Discriminative
Probabilistic Classifier

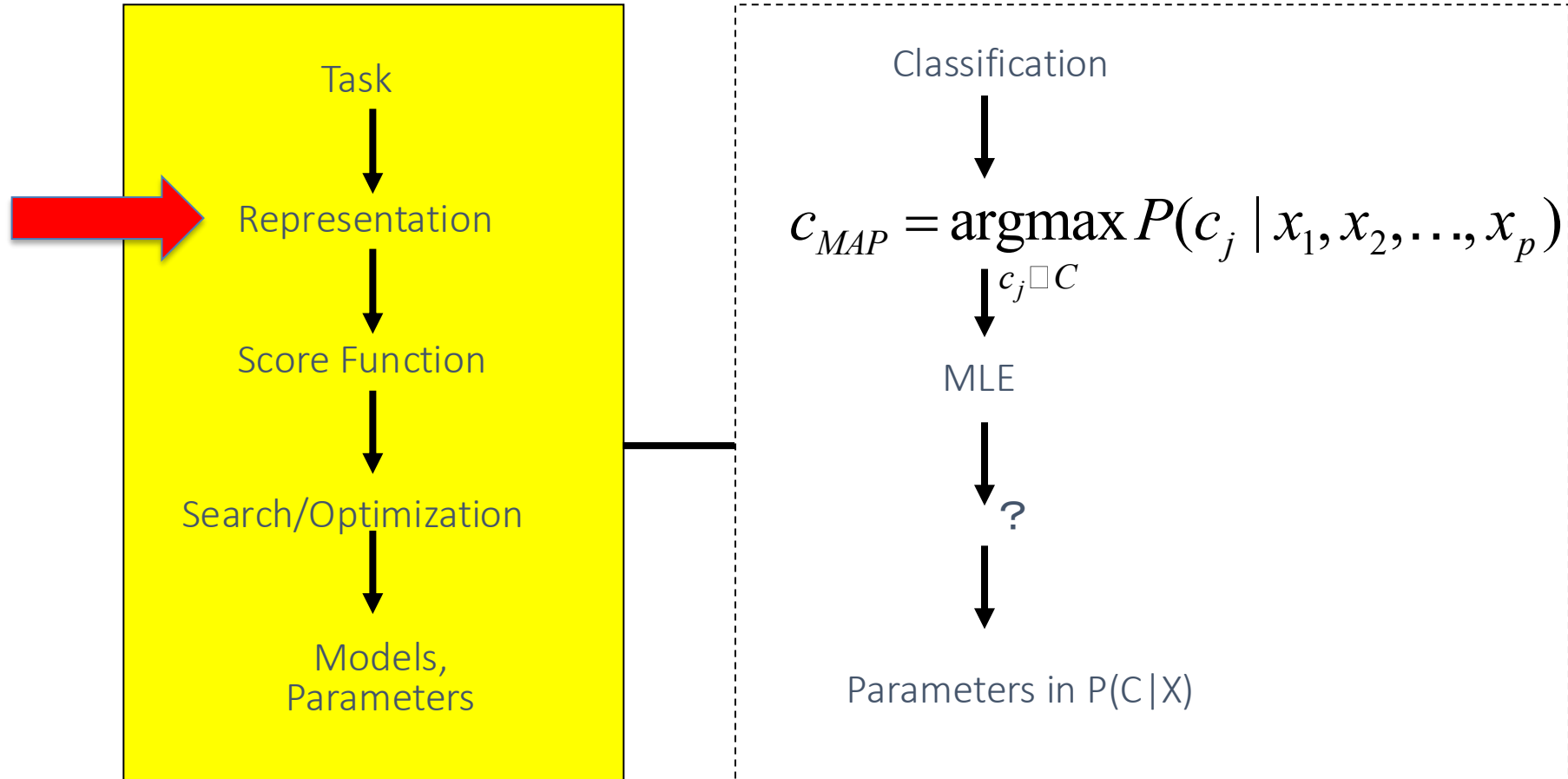
$$x_1 \quad x_2 \quad \dots \quad x_p$$

$$\mathbf{x} = (x_1, x_2, \dots, x_p)$$

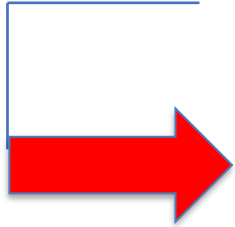
$$P(c|\bar{x})$$



Bayes Classifier



Roadmap



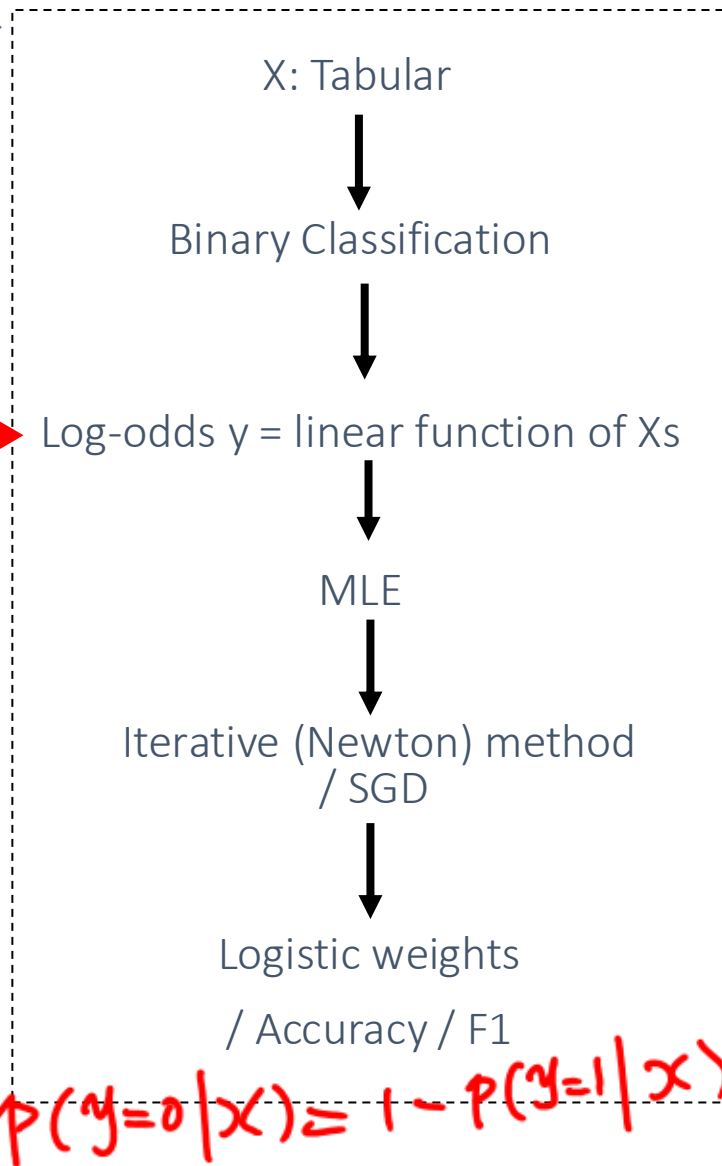
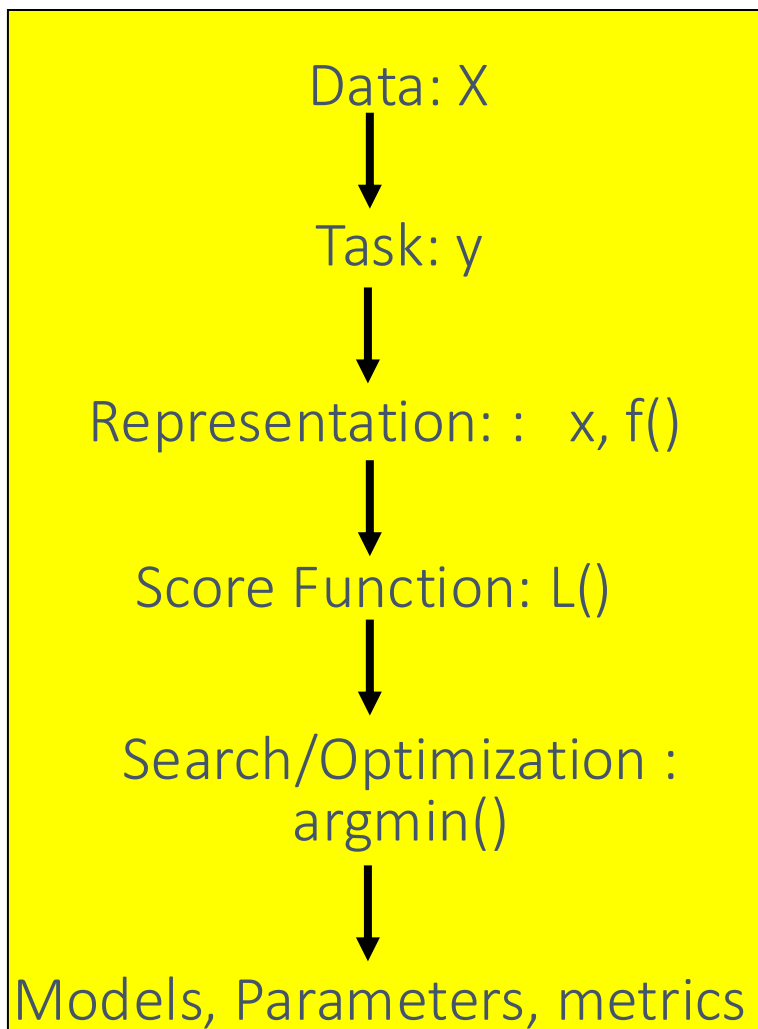
- ☐ Bayes Classifier: Classification MAP Rule
- ☐ Logistic Regression Representation
- ☐ Training LG by MLE

Summary: Logistic Regression Classifier

- View I: $\text{logit}(y)$ as linear of X s
- View II: model Y as Bernoulli with $p(y=1|x)$ as $p(\text{Head})$
- View III: S" shape function compress to $[0,1]$
- View IV: models a linear classification boundary!
- View V: Two stages: summation + sigmoid

Today: Logistic Regression **Classifier**

$$P(y = 1|x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$



Multivariate linear regression to Logistic Regression

$$\underline{y} = \underline{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}$$

Logistic regression for
binary classification

Log-odds

$$\ln \left[\frac{P(y|x)}{1 - P(y|x)} \right] = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Multivariate linear regression to Logistic Regression

$$\underline{y} = \underline{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}$$

Logistic regression for
binary classification

Binary Variable
 y : Yes, No
1, 0
H, T

Log-odds

$$\ln \left[\frac{P(y|x)}{1 - P(y|x)} \right] = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

$$P(y=0|x) = 1 - P(y=1|x)$$

View I: logit of $p(y=1|x)$ is linear function of x

$$\ln\left(\frac{P}{1-P}\right) \xrightarrow{P=P(y=1|x)} \ln\left[\frac{P(y|x)}{1-P(y|x)}\right] = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

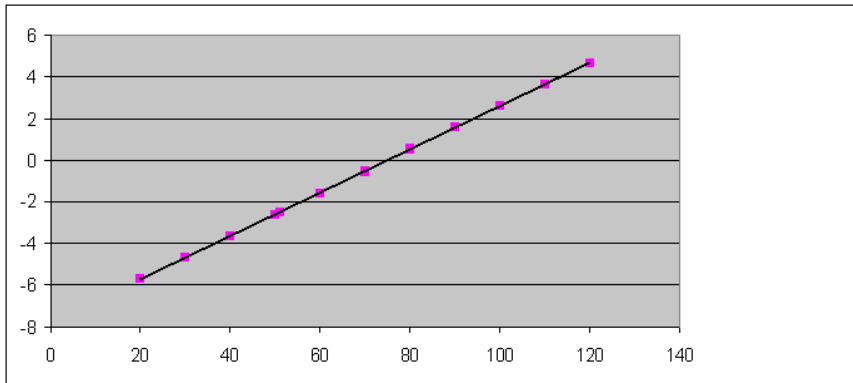
Logit function Logit of $P(y|x)$

Summary: Logistic Regression Classifier

- View I: $\text{logit}(y)$ as linear of X s
- View II: model Y as Bernoulli with $p(y=1 | x)$ as $p(\text{Head})$
- View III: S" shape function compress to $[0,1]$
- View IV: models a linear classification boundary!
- View V: Two stages: summation + sigmoid

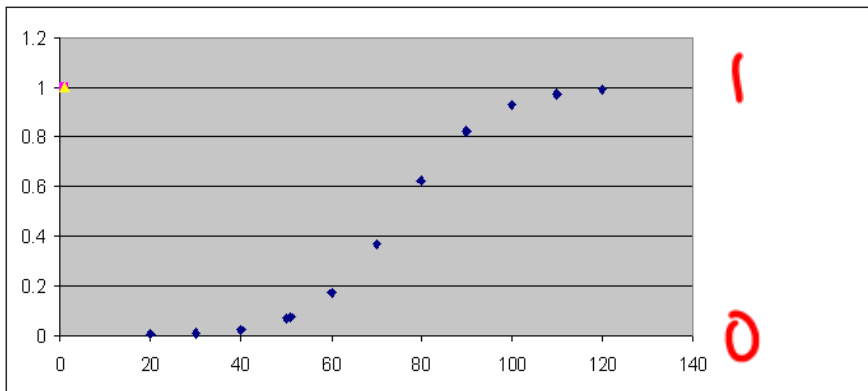
View II: Model $Y|X$ as Bernoulli:

$\ln[p/(1-p)]$



$P(Y=1|x)$

S
shape



Bernoulli Distribution

p_{Head}
↓

$Y \in \{0, 1\}$

$$p_{\text{Head}} = P(Y=1|x) = \frac{e^{\alpha + \beta x}}{1 + e^{\alpha + \beta x}}$$



$P(Y=1|x)$ $1 - P(Y=1|x)$

Logistic Regression $p(y|x)$

$$\ln \left[\frac{P(y|x)}{1 - P(y|x)} \right] = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Logit



P(Head)

$$P(y|x) = \frac{e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}}{1 + e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}} = \frac{1}{1 + e^{-(\beta_0 + \beta^T X)}}$$

Review: Bernoulli Distribution

e.g. Coin Flips

- You flip a coin

Bernoulli(p)

$\{H, T\}$

- Z : {Who is Up: Head or Tail} is a discrete Random Variable

- Head with probability $p = p(\text{Head})$

- **Binary** random variable

- **Bernoulli trial** with success probability p

$\{z_1, z_2, \dots, z_n\}$

$\{H \ H \ T \ \dots \ H\} \xrightarrow{\text{MLE}} p(\text{Head})$

Logistic Regression—Bernoulli Distribution with $p(\text{Head})$ as a function of x

Logistic regression models are appropriate when the target variable is coded as 0/1.

We only observe “0” and “1” for the target variable—
but we think of the target variable conceptually as a
probability that “class=1” will occur.

$$p(\text{Head}) = p(y=1|x) = \frac{e^{\beta^T x}}{e^{\beta^T x} + 1}$$

This means we use Bernoulli distribution to model the target variable with its Bernoulli parameter $p=p(y=1|x)$ predefined.

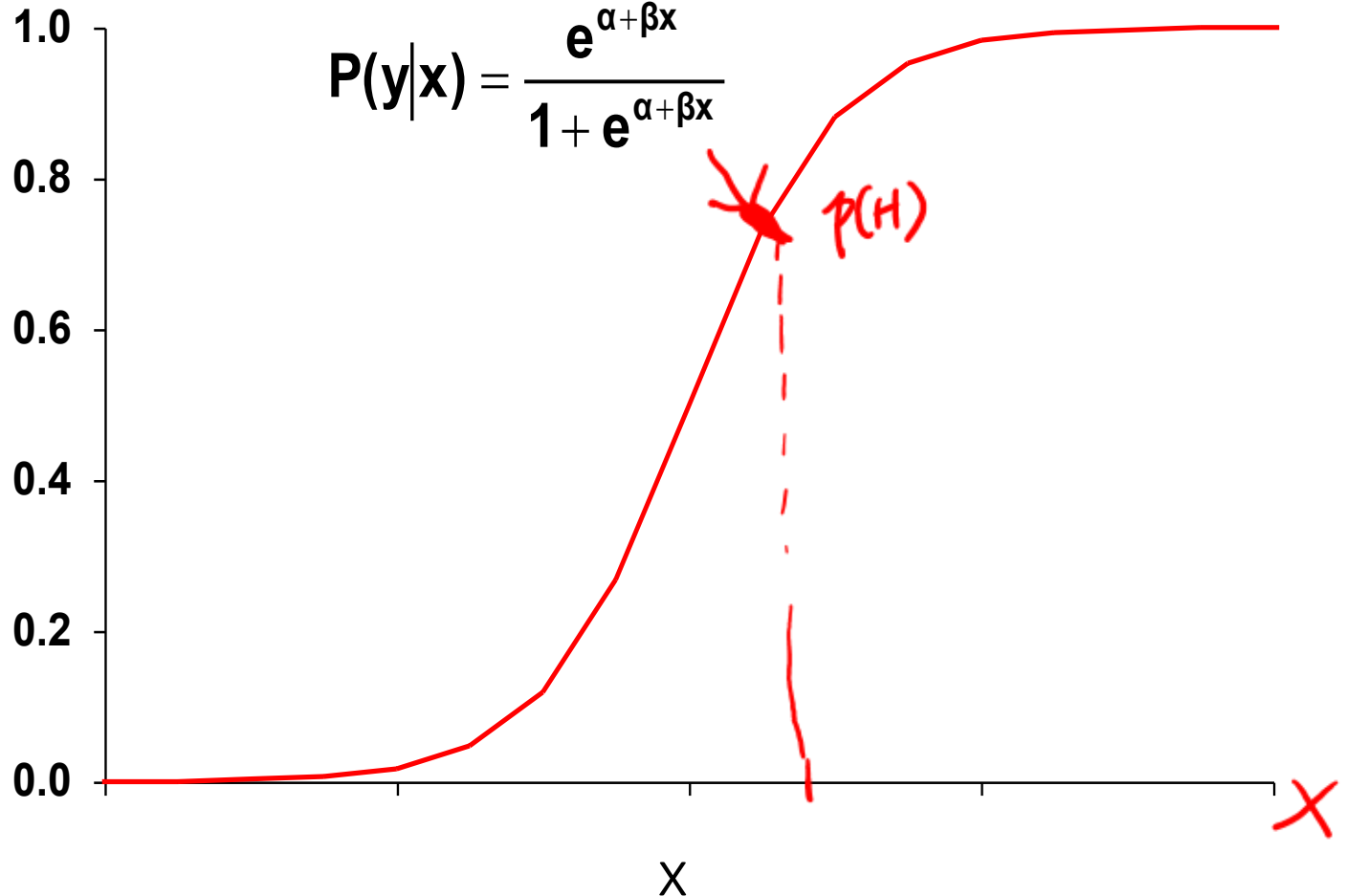
The main interest → predicting the probability that an event occurs (i.e., the probability that $p(y=1|x)$).

View II: Model $Y|X$ as Bernoulli:

e.g.
Probability of
disease

$P(Y=1|X)$

$$P(y|x) = \frac{e^{\alpha + \beta x}}{1 + e^{\alpha + \beta x}}$$



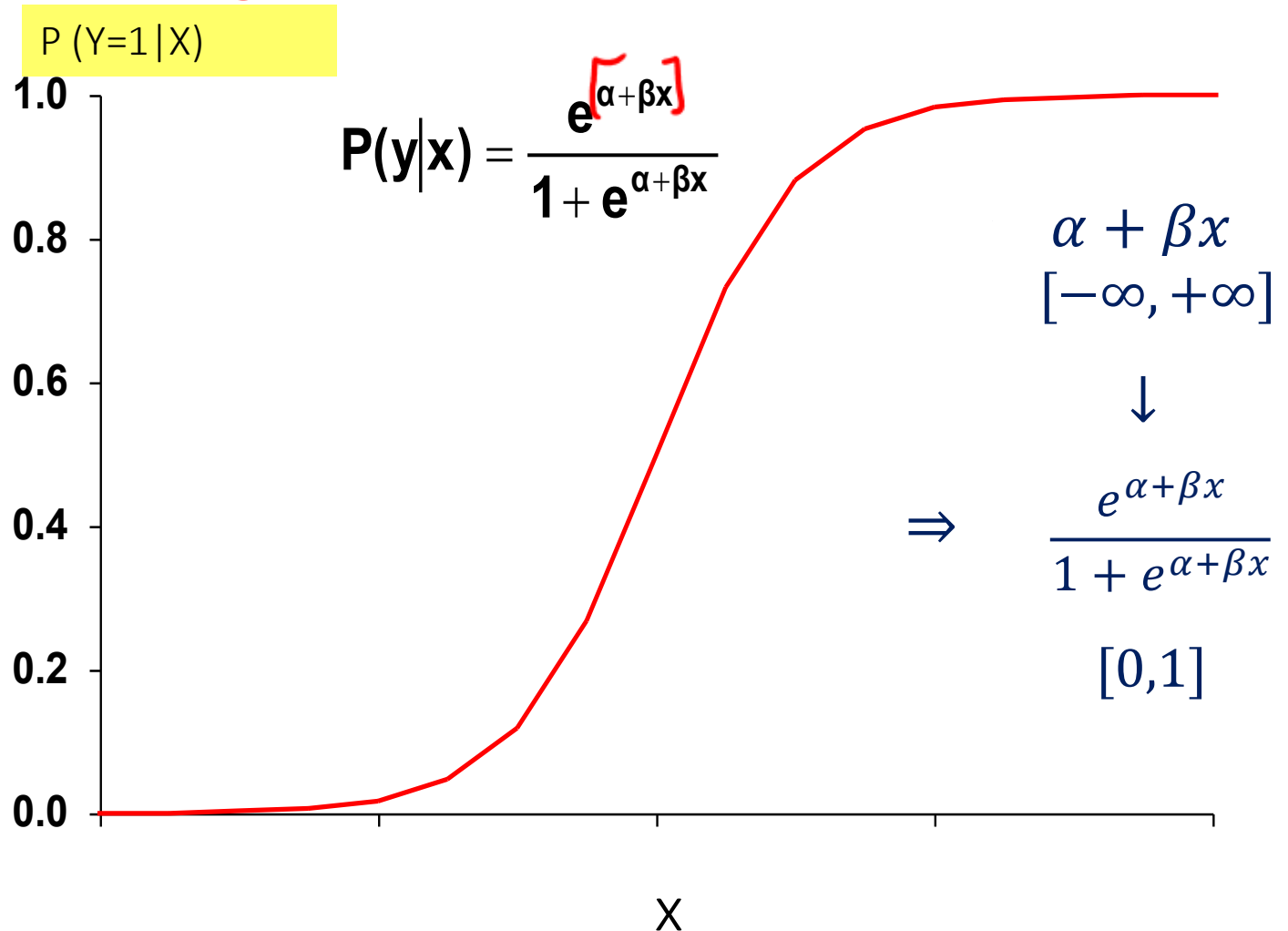
Summary: Logistic Regression Classifier

- View I: $\text{logit}(y)$ as linear of X s
- View II: model Y as Bernoulli with $p(y=1 | x)$ as $p(\text{Head})$
- View III: S" shape function compress to $[0,1]$
- View IV: models a linear classification boundary!
- View V: Two stages: summation + sigmoid

View III: "S" shape function compress output to [0,1]

LogisticR = LR + Sigmoid

Sigmoid
 $\frac{e^h}{1+e^h}$

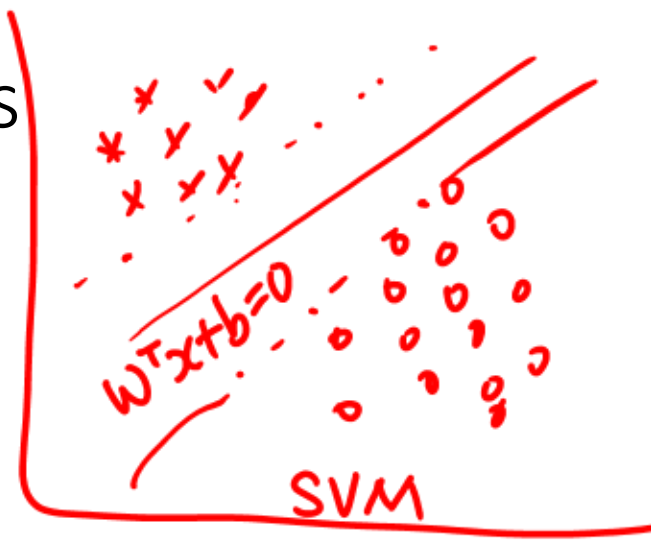


View IV: Logistic Regression models a linear classification boundary!

$$y = \begin{Bmatrix} 1 \\ 0 \end{Bmatrix}$$

$$\underset{y \in \{0,1\}}{\operatorname{argmax}} p(y|x)$$

$$\frac{p(y=0|x) = p(y=1|x)}{\log\left(\frac{p(y=1|x)}{p(y=0|x)}\right) = \beta^T x = \log(1) = 0} \Rightarrow \text{Decision Boundary} \frac{p(y=1|x)}{p(y=0|x)} = 1$$



View IV: Logistic Regression models a linear classification boundary!

$$y \in \{0,1\}$$

$$\ln \left[\frac{P(y|x)}{1 - P(y|x)} \right] = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Decision Boundary $\rightarrow$ equals to zero

$$\ln \left[\frac{P(y=1|x)}{P(y=0|x)} \right] = \ln \left[\frac{P(y=1|x)}{1 - P(y=1|x)} \right] = \alpha + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Thank You



UVA CS 4774: Machine Learning

Lecture 11: Logistic Regression

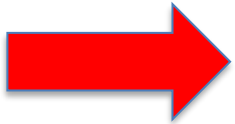
Module II

Dr. Yanjun Qi

University of Virginia
Department of Computer Science

Roadmap

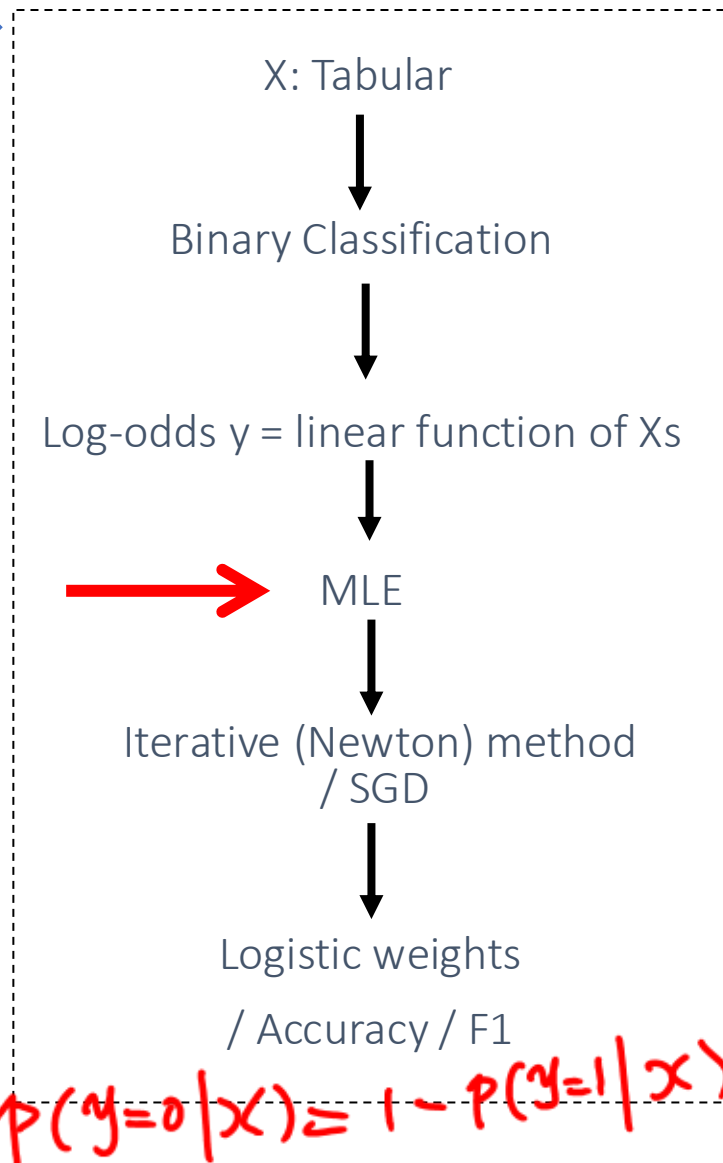
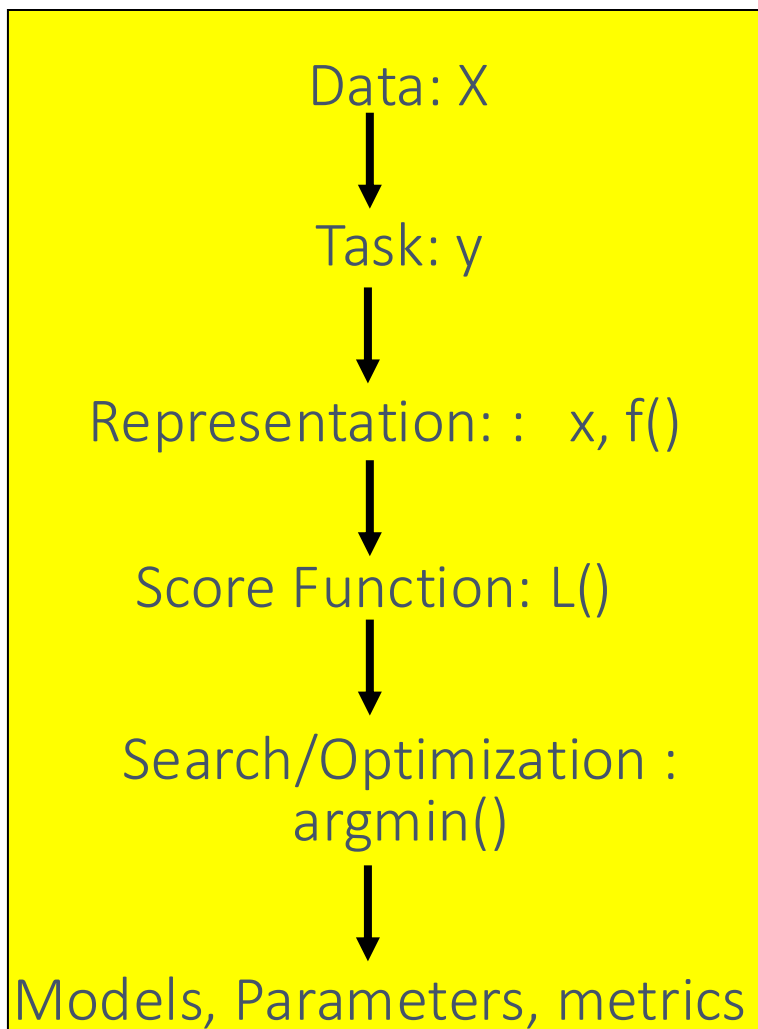
- ☐ Bayes Classifier: Classification MAP Rule
- ☐ Logistic Regression Representation
- ☐ Training LG by MLE



$$\begin{aligned} & p(c | \vec{x}) \\ & \downarrow \\ & p(y=1 | \vec{x}) = \frac{e^{\beta^T \vec{x}}}{e^{\beta^T \vec{x}} + 1} \end{aligned}$$

Today: Logistic Regression **Classifier**

$$P(y = 1|x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$



Maximum Likelihood Estimation

$\{H H T T \dots H\}$

$Z: \{H, T\} \rightarrow \{Z_1, Z_2, \dots, Z_n\}$
 $P(Z): \{p, 1-p\}$
 $\downarrow P(\text{Head})$

A general Statement

Consider a sample set $T=(Z_1 \dots Z_n)$ which is drawn from a probability distribution $P(Z|\theta)$ where θ are parameters.

If the Z s are independent with probability density function $P(Z_i|\theta)$, the joint probability of the whole set is

$$\theta^* = \arg \max_{\theta} \left\{ P(Z_1 \dots Z_n | \theta) \right\} = \prod_{i=1}^n \underbrace{P(Z_i | \theta)}_{p(\text{Head})}$$

this may be maximised with respect to θ to give the maximum likelihood estimates.

The idea is to

- ✓ assume a particular model with unknown parameters, θ
- ✓ we can then define the probability of observing a given event conditional on a particular set of parameters. $P(Z_i|\theta)$
- ✓ We have observed a set of outcomes in the real world. $z_1, z_2, \dots, z_n$
- ✓ It is then possible to choose a set of parameters which are most likely to have produced the observed results.

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} P(Z_1 \dots Z_n | \theta) = \prod_{i=1}^n P(Z_i | \theta)$$

Likelihood

$0 < < 1$

This is maximum likelihood. In most cases it is both consistent and efficient.

$$\log(L(\theta)) = \sum_{i=1}^n \log(P(Z_i | \theta))$$

Log-Likelihood

It is often convenient to work with the Log of the likelihood function.

Review: Defining Likelihood for **basic** Bernoulli

Given: $\{z_1, z_2, \dots, z_n\}$

$\Downarrow$
 $\{H, H, T, \dots, H\}_n$

$\Downarrow$ reformulate

$\{1, 1, 0, \dots, 1\}_n$

$$[p(z_i | \underline{\theta})] = \underbrace{p^{z_i} (1-p)^{1-z_i}}_{\text{Here } z_i \in \{0, 1\}}$$

$$p(z_i) = \begin{cases} p, & \text{if } z_i = H/1 \\ 1-p, & \text{if } z_i = T/0 \end{cases}$$

$$\Rightarrow \underset{p_{\text{Head}}}{\text{argmax}} \prod_{i=1}^n p^{z_i} (1-p)^{1-z_i}$$

constant
 $\theta = \{p\}$
 $= \{p(\text{Head})\}$

Review: Defining Likelihood for **basic** Bernoulli

$$\begin{aligned}\log(L(p)) &= \log \left[\prod_{i=1}^n p^{z_i} (1-p)^{1-z_i} \right] \\ &= \sum_{i=1}^n (z_i \log p + (1-z_i) \log(1-p)) \\ &= \log p \sum_{i=1}^n z_i + \log(1-p) \sum_{i=1}^n (1-z_i) \\ &= x \log p + (n-x) \log(1-p)\end{aligned}$$

Review: Defining Likelihood for **basic** Bernoulli

$$\underset{p}{\operatorname{argmax}} \{-l(p)\} = \underset{p}{\operatorname{argmin}} \left\{ -x \log(p) - (n-x) \log(1-p) \right\} \quad \text{i.e. negative log-likelihood}$$

$$\frac{dl(p)}{dp} = 0 - \frac{x}{p} - \frac{-(n-x)}{1-p} = 0$$

$$0 = -x + pn$$

Minimize the negative log-likelihood

→ MLE parameter estimation

$$0 = -\frac{x}{p} + \frac{n-x}{1-p}$$

$$0 = \frac{-x(1-p) + p(n-x)}{p(1-p)}$$

$$0 = -x + px + pn - px$$

$$\hat{p} = \frac{x}{n} \quad \text{i.e. Relative frequency of a binary event}$$

The proportion of heads!

From Basic Bernoulli to Logistic Regression

Observing binary samples z_i

PMF:

$$\Pr(z_i | p) = p^{z_i} (1 - p)^{1 - z_i}$$

LIKELIHOOD:

$$L(p) = \prod_{i=1}^n p^{z_i} (1 - p)^{1 - z_i}$$

↑
function of $p = \Pr(\text{head})$

$$\{H, H, T, \dots, H\}_n$$

Logistic Regression $z_i = y_i | x_i$

$$\{y_1/x_1, y_2/x_2, \dots, y_n/x_n\}$$

$$p(z_i | \beta)$$

$$p(y_i = 1 | \beta) = \frac{e^{\beta^T \vec{x}}}{e^{\beta^T \vec{x}} + 1}$$

Now we just rewrite

$$\hat{y}_i = p(y_i = 1 | x_i)$$

$$p(z_i | \beta) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1 - y_i}$$

LIKELIHOOD:

Basic Bernoulli

$$L(p) = \prod_{i=1}^n p^{z_i} (1-p)^{1-z_i}$$

↑
function of $p = \text{Pr}(\text{head})$

$$\begin{aligned} \log(L(p)) &= \log \left[\prod_{i=1}^n p^{z_i} (1-p)^{1-z_i} \right] \\ &= \sum_{i=1}^n (z_i \log p + (1-z_i) \log(1-p)) \end{aligned}$$

Logistic / Bernoulli

$$L(\beta)$$

$$= \prod_{i=1}^n p(y_i=1|x_i)^{y_i} (1-p(y_i=1|x_i))^{1-y_i}$$

$$= \prod_{i=1}^n \hat{y}_i^{y_i} (1-\hat{y}_i)^{1-y_i}$$

Loglikelihood [Cross Entropy]

$$\ell(\beta) = \sum_{i=1}^n \left(y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i) \right)$$

$$\text{ll}(\beta) = \sum_{i=1}^N \{\log \Pr(Y = y_i | X = x_i)\}$$

When training set includes (x_i, y_i) , $i=1, \dots, N$

$$\text{ll}(\beta) = \sum_{i=1}^N \log P(y_i | x_i)$$

$$\text{Here } P(y_i | x_i) = \begin{cases} p(y=1 | x_i), & \text{if } y_i = 1 \\ p(y=0 | x_i), & \text{if } y_i = 0 \end{cases}$$

$$= (p(y=1 | x_i))^{y_i} (1 - p(y=1 | x_i))^{1-y_i}$$

MLE for Logistic Regression Training

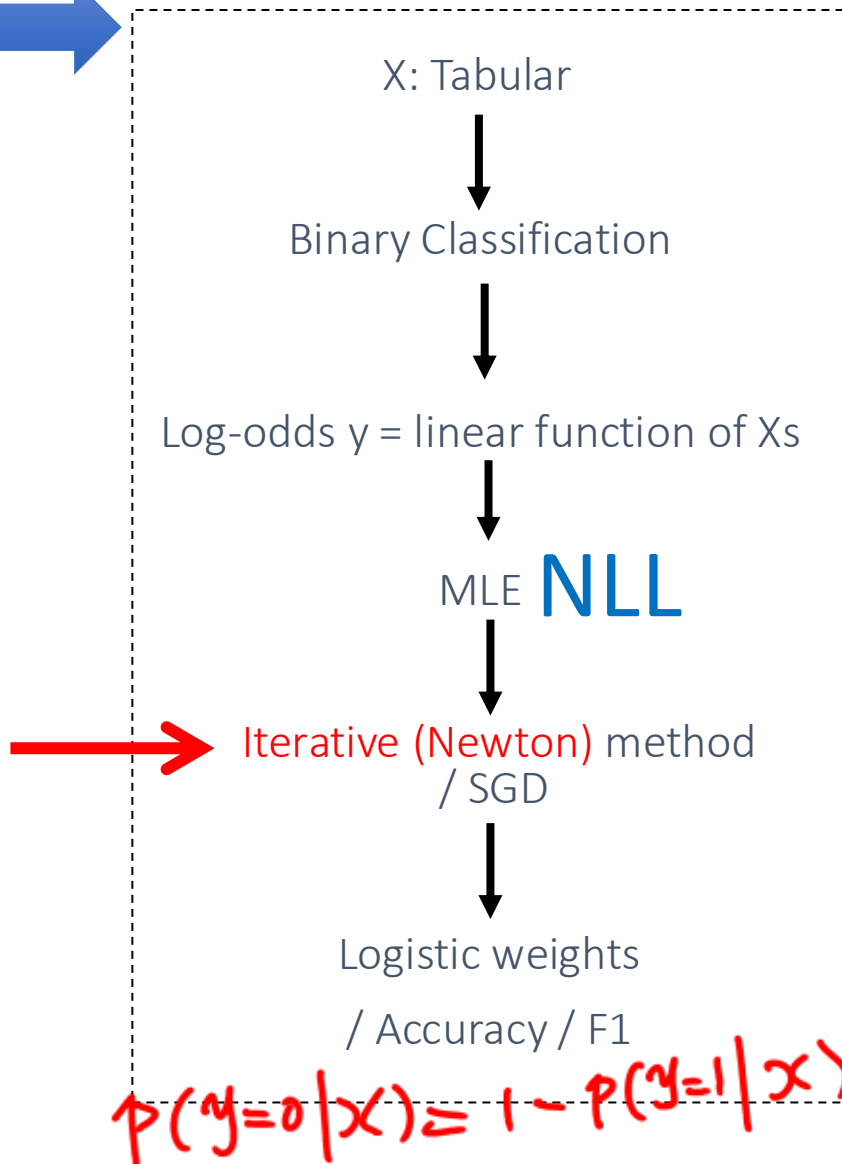
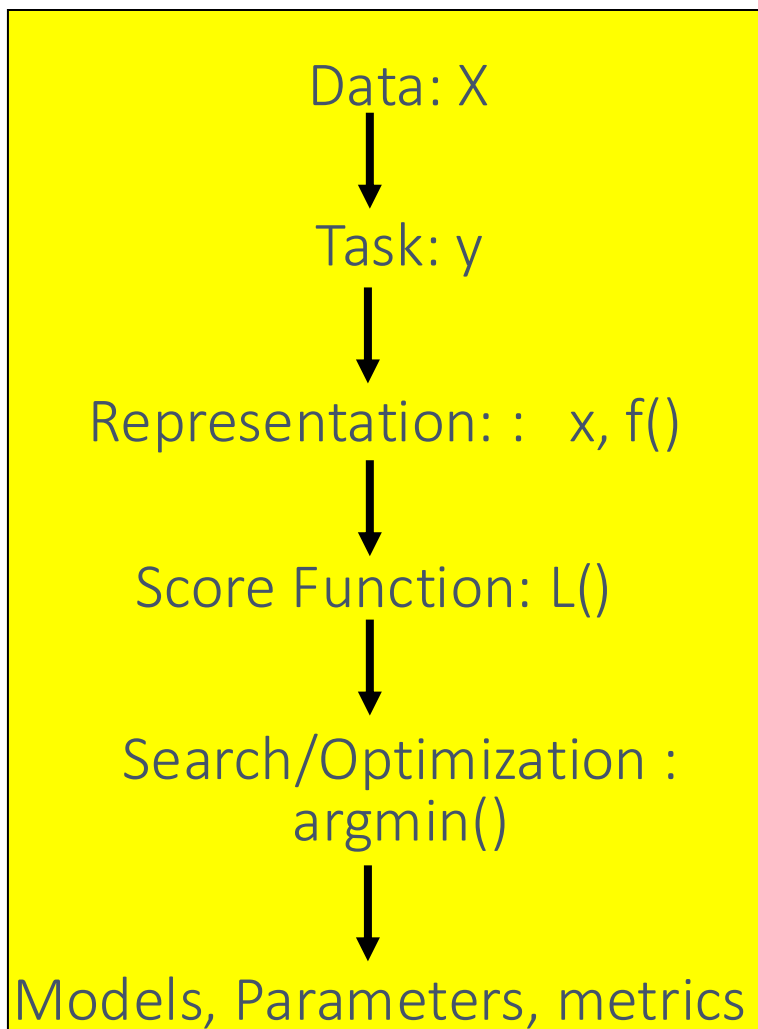
Training set: $(x_i, y_i), i=1, \dots, N$

$$\begin{aligned} l(\beta) &= \sum_{i=1}^N \{\log \Pr(Y = y_i | X = x_i)\} \\ &= \sum_{i=1}^N \{y_i \log(\Pr(Y = 1 | X = x_i)) + (1 - y_i) \log(\Pr(Y = 0 | X = x_i))\} \\ &= \sum_{i=1}^N \left(y_i \log \frac{\exp(\beta^T x_i)}{1 + \exp(\beta^T x_i)} + (1 - y_i) \log \frac{1}{1 + \exp(\beta^T x_i)} \right) \\ &= \sum_{i=1}^N (y_i \beta^T x_i - \log(1 + \exp(\beta^T x_i))) \end{aligned}$$

i.e. negative cross entropy

Today: Logistic Regression **Classifier**

$$P(y = 1|x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$



Summary: MLE for Logistic Regression Training

Let's fit the logistic regression model for $K=2$, i.e., number of classes is 2

Training set: $(x_i, y_i), i=1, \dots, N$

(conditional)
Log-likelihood:

How?

For Bernoulli distribution

$$p(y | x)^y (1 - p)^{1-y}$$

$$\begin{aligned} l(\beta) &= \sum_{i=1}^N \{\log \Pr(Y = y_i | X = x_i)\} \\ &= \sum_{i=1}^N y_i \log(\Pr(Y = 1 | X = x_i)) + (1 - y_i) \log(\Pr(Y = 0 | X = x_i)) \\ &= \sum_{i=1}^N \left(y_i \log \frac{\exp(\beta^T x_i)}{1 + \exp(\beta^T x_i)} + (1 - y_i) \log \frac{1}{1 + \exp(\beta^T x_i)} \right) \\ &= \sum_{i=1}^N (y_i \beta^T x_i - \log(1 + \exp(\beta^T x_i))) \end{aligned}$$

x_i are $(p+1)$ -dimensional input vector with leading entry 1

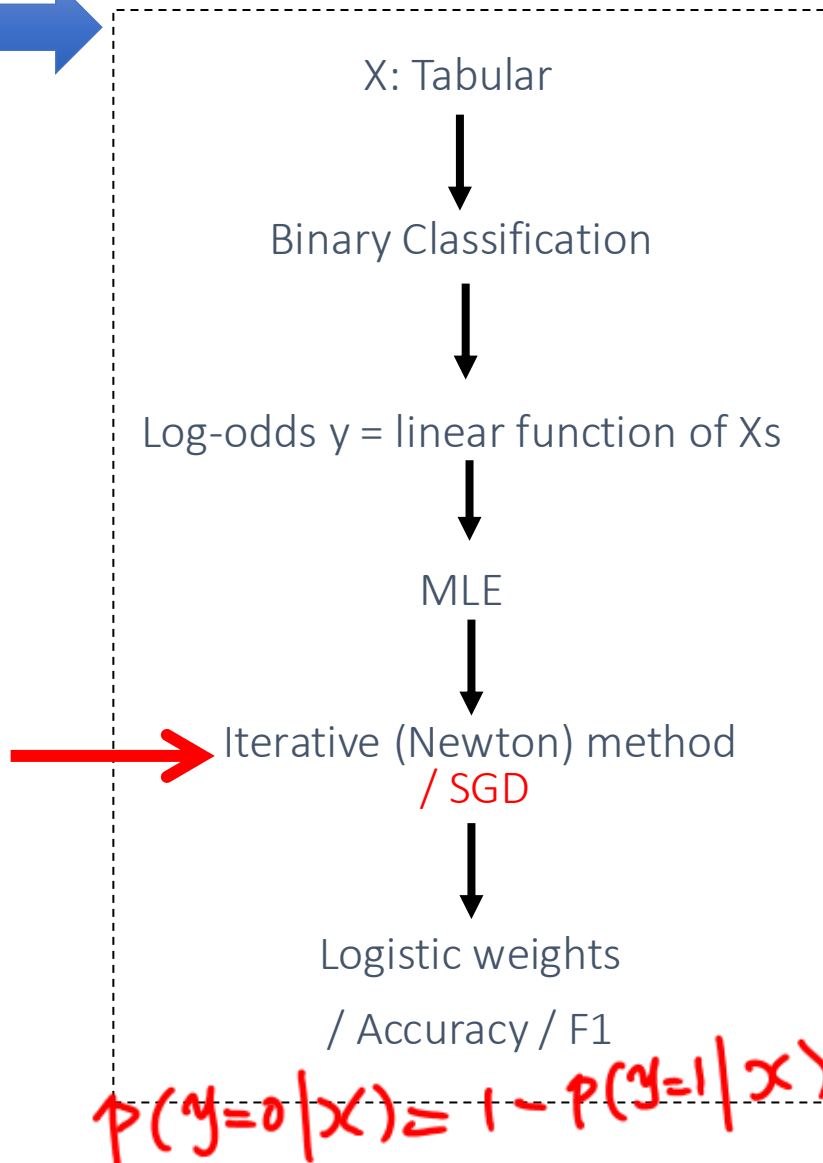
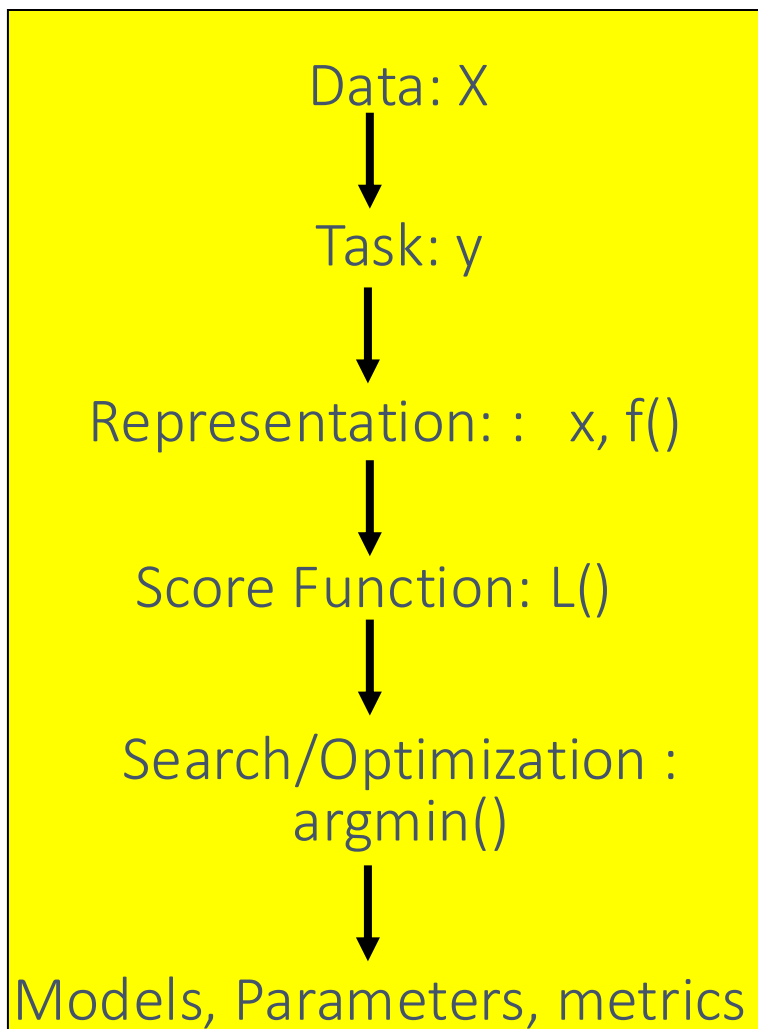
β is a $(p+1)$ -dimensional vector

We want to **maximize** the log-likelihood in order to estimate β

See Extra Slides How to use Newton-Raphson optimization

Today: Logistic Regression **Classifier**

$$P(y = 1|x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$



$$p(y=0|x) = 1 - p(y=1|x)$$

ReWrite Logistic Regression as two stages:

VIEW II

First:

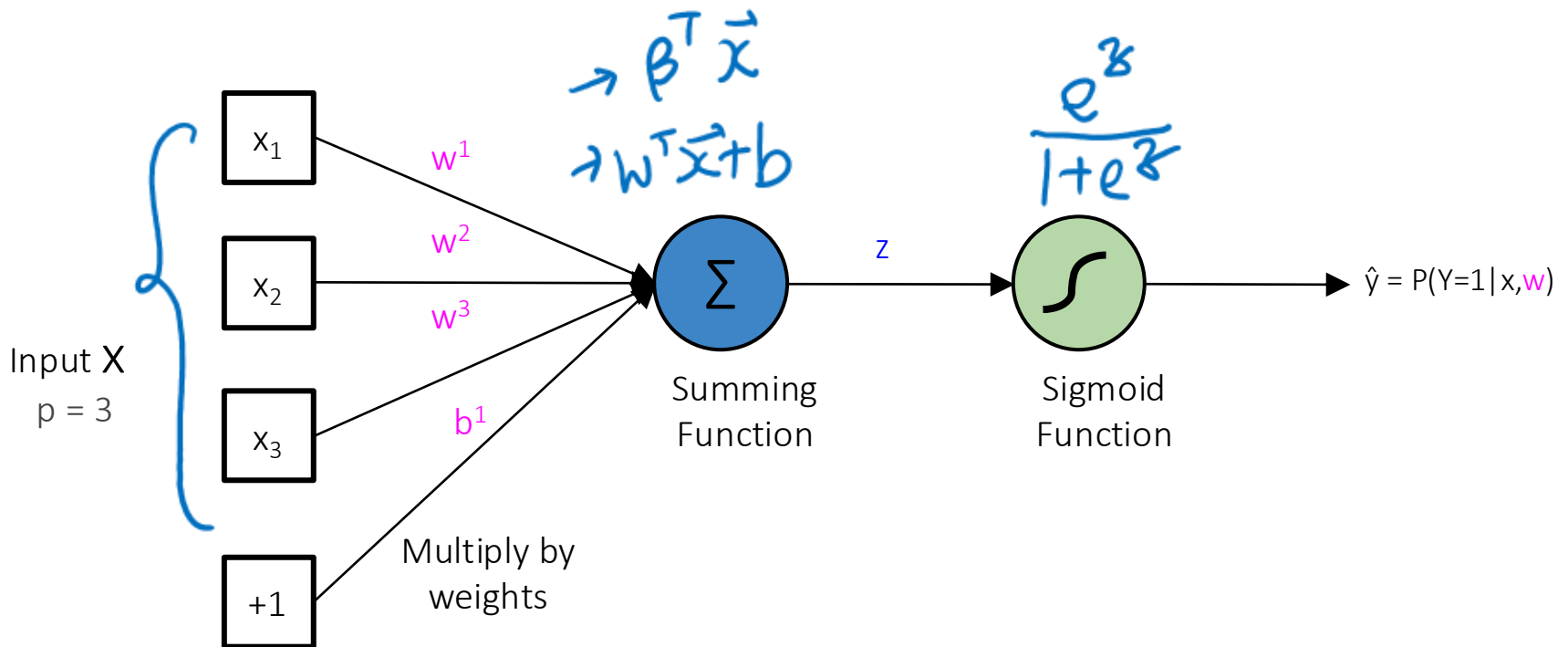
Summing $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$

Second:

Sigmoid
Squashing

$$\hat{y} = P(y=1|x) = \frac{e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}}{1 + e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p}} = \frac{e^z}{1 + e^z}$$

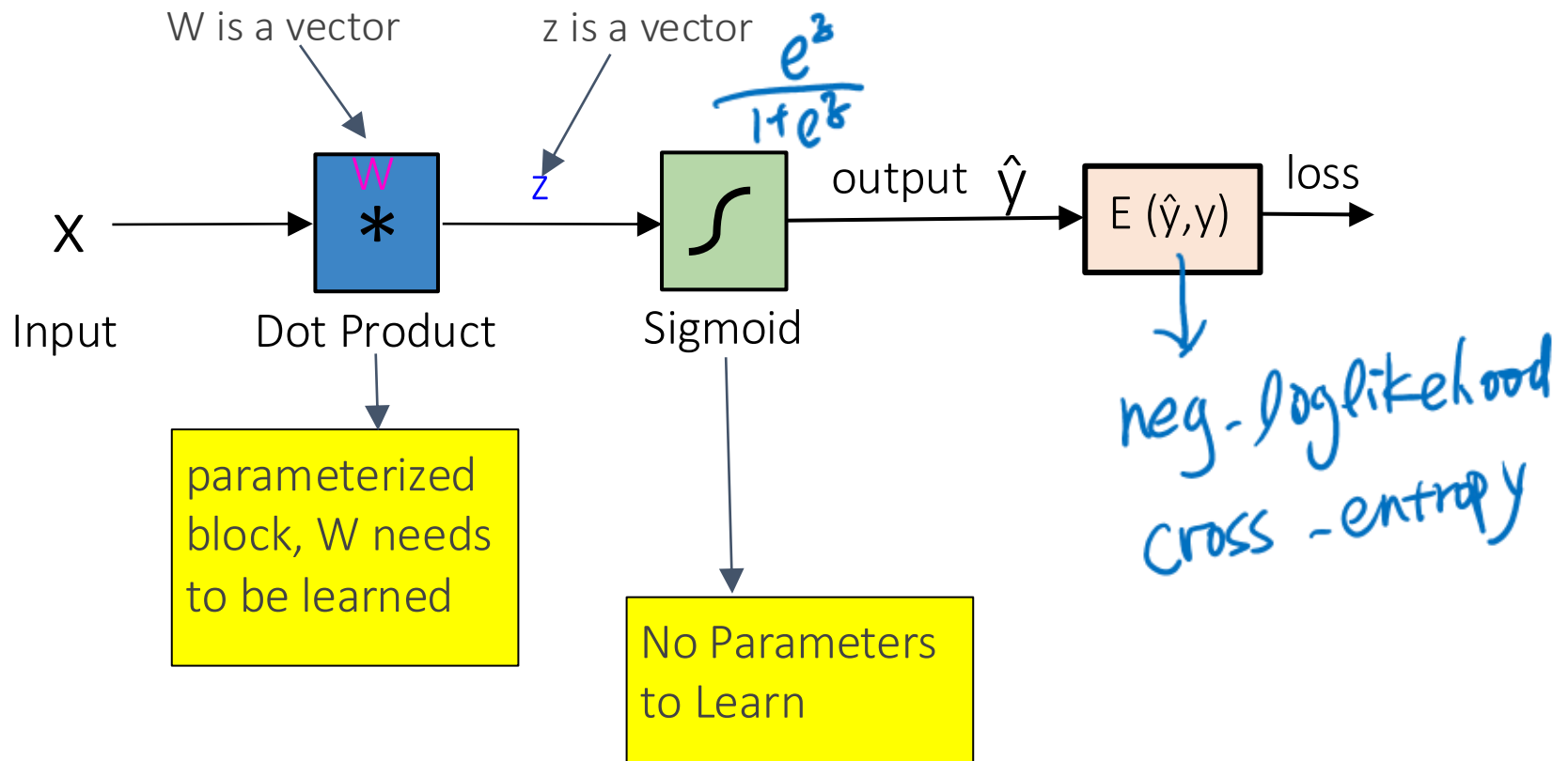
One “Neuron”: Block View of Logistic Regression



$$z = w^T \cdot X + b$$

$$y = \text{sigmoid}(z) = \frac{e^z}{1 + e^z}$$

Next: “Block View” of Logistic Regression (plus loss)



Next: Stochastic GD →

- For LR: linear regression, We have the following gradient descent rule:

$$\theta_j^{t+1} = \theta_j^t - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \Big|_t$$

$J(\theta) \downarrow$
 $\Downarrow$
 θ

- For neural network, we have the delta rule

$$\Delta \mathbf{w} = -\eta \frac{\partial E}{\partial \mathbf{W}^t}$$

$$\mathbf{W}^{t+1} = \mathbf{W}^t - \eta \frac{\partial E}{\partial \mathbf{W}^t} = \mathbf{W}^t + \Delta \mathbf{w}$$

$E(\mathbf{w}) \downarrow$
 $\Downarrow$
 $\mathbf{w}$

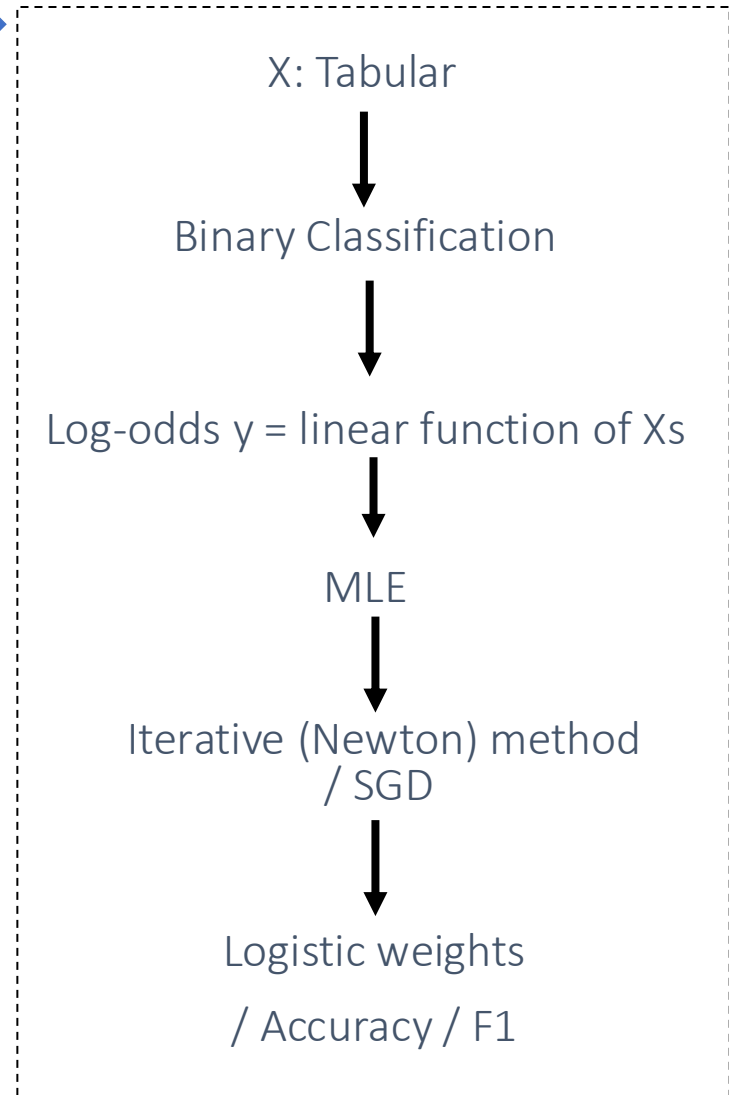
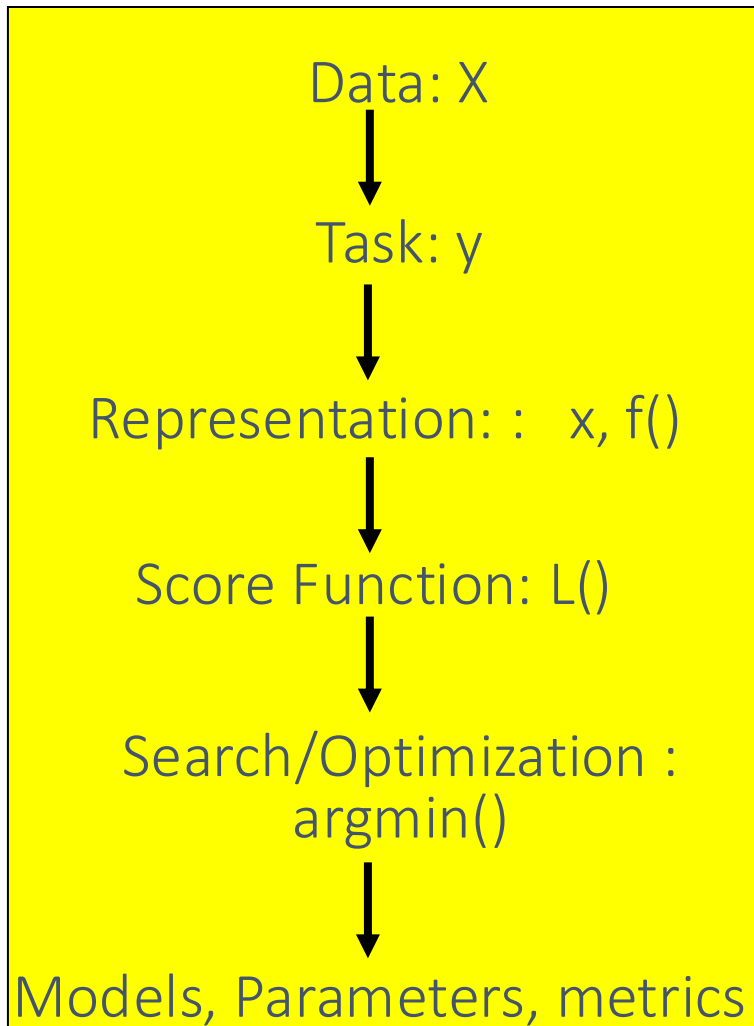
Takeaway: Logistic Regression Classifier

- View I: $\text{logit}(y)$ as linear of X s
- View II: model Y as Bernoulli with $p(y=1|x)$ as $p(\text{Head})$
- View III: S" shape function compress to $[0,1]$
- View IV: models a linear classification boundary!
- View V: Two stages: summation + sigmoid

Today: Logistic Regression **Classifier**

$$P(y = 1|x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$

p(Head)



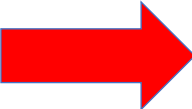
Thank You



References

- Prof. Tan, Steinbach, Kumar's "Introduction to Data Mining" slide
- Prof. Andrew Moore's slides
- Prof. Eric Xing's slides
- Prof. Ke Chen NB slides
- Hastie, Trevor, et al. *The elements of statistical learning*. Vol. 2. No. 1. New York: Springer, 2009.

Today: Extra

- 
- ✓ Bayes Classifier and MAP Rule?
 - Bayes Classifier
 - Expected Prediction Error
 - 0-1 Loss function for Bayes Classifier

- ✓ Logistic regression
 - Parameter Estimation for LR

$$p(y|x) = \frac{e^{\beta x}}{1 + e^{\beta x}}$$

$\Downarrow$
 $\frac{1}{\beta}$

Bayes classifiers

- Treat each feature attribute and the class label as random variables.
- Given a sample x with attributes $(x_1, x_2, \dots, x_p)$:
 - Goal is to predict its class C .
 - Specifically, we want to find the value of C_i that maximizes $p(C_i | x_1, x_2, \dots, x_p)$.
- Can we estimate $p(C_i | x) = p(C_i | x_1, x_2, \dots, x_p)$ directly from data?

Bayes classifiers

- Treat each feature attribute and the class label as random variables.
- Given a sample x with attributes $(x_1, x_2, \dots, x_p)$:
 - Goal is to predict its class C .
 - Specifically, we want to find the value of C_i that maximizes $p(C_i | x_1, x_2, \dots, x_p)$.
- Can we estimate $p(C_i | x) = p(C_i | x_1, x_2, \dots, x_p)$ directly from data?

$\rightarrow \{C_1, C_2, \dots, C_L\}$

Bayes classifiers

→ MAP classification rule

- Establishing a probabilistic model for classification

→ MAP classification rule

- MAP: Maximum A Posterior
- Assign x to c^* if

$$P(C = c^* | \mathbf{X} = \mathbf{x}) \geq P(C = c | \mathbf{X} = \mathbf{x})$$

for $c^* \in \{c_1, \dots, c_L\}$

Bayes classifiers

→ MAP classification rule

- Establishing a probabilistic model for classification

→ MAP classification rule

- MAP: Maximum A Posterior
- Assign x to c^* if

$$P(C = c^* | \mathbf{X} = \mathbf{x}) > P(C = c | \mathbf{X} = \mathbf{x})$$

$$c \neq c^*, c = c_1, \dots, c_L$$

$$\left\{ \begin{array}{l} P(C = c_1 | \mathbf{x}) \\ P(C = c_2 | \mathbf{x}) \\ P(C = c_3 | \mathbf{x}) \end{array} \right\} \max \Rightarrow c_i$$

Bayes Classifiers – MAP Rule

Task: Classify a new instance X based on a tuple of attribute values into one of the classes

$$X = \langle X_1, X_2, \dots, X_p \rangle$$

$$c_{MAP} = \operatorname{argmax}_{c_j \in C} P(c_j \mid x_1, x_2, \dots, x_p)$$



WHY ?

MAP = Maximum A posteriori Probability

Recap: Statistical Decision Theory (Extra)

- Random input vector: X
- Random output variable: Y
- Joint distribution: $\Pr(X, Y)$
- Loss function $L(Y, f(X))$

$$\Rightarrow D = \begin{pmatrix} (\vec{x}_1, y_1) \\ \vdots \\ (\vec{x}_n, y_n) \end{pmatrix}$$

- Expected prediction error (EPE):

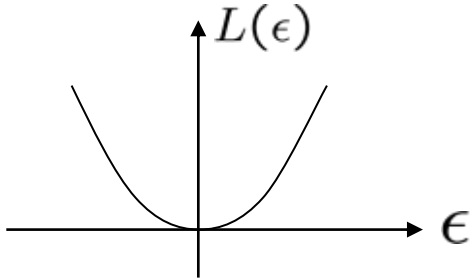
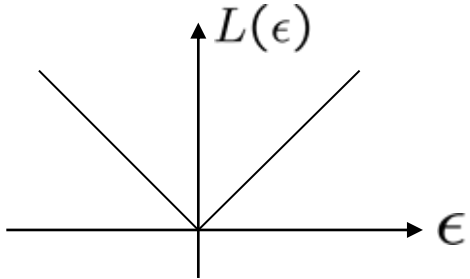
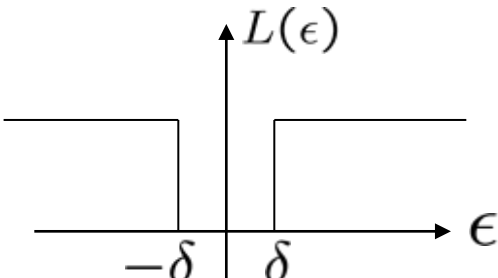
$$\text{EPE}(f) = E(L(Y, f(X))) = \int L(y, f(x)) \Pr(dx, dy)$$

$$\text{e.g.} = \int (y - f(x))^2 \Pr(dx, dy)$$

e.g. Squared error loss (also called L2 loss)

Consider
population
distribution

SUMMARY: WHEN Expected prediction error (EPE) USES DIFFERENT LOSS (Extra)

Loss Function	Estimator $\hat{f}(x)$
L_2 	$\hat{f}(x) = E[Y X = x]$
L_1 	$\hat{f}(x) = \text{median}(Y X = x)$
$0-1$ 	$\hat{f}(x) = \arg \max_Y P(Y X = x)$ (Bayes classifier / MAP)

0-1 LOSS for Classification

$$\text{if } k = l, L(k, l) = 0$$

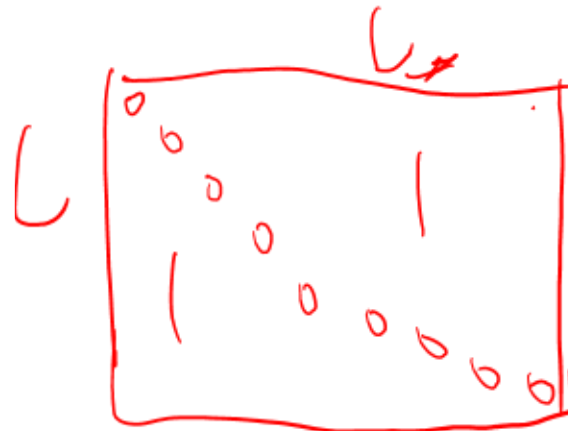
- Procedure for categorical output variable C

$$\text{if } k \neq l, L(k, l) = 1$$

- Frequently, 0-1 loss function used: $L(k, l)$

- $L(k, l)$ is the price paid for misclassifying an element from class C_k as belonging to class C_l

→ $L \times L$ matrix



$C_1, C_2, \dots, C_L$

Expected prediction error (EPE)

- Expected prediction error (EPE), with expectation taken w.r.t. the **joint distribution** $\Pr(C, X)$

- $\Pr(C, X) = \Pr(C | X) \Pr(X)$

→ e.g. 0-1 loss

$$E_x(X)$$

$$E_x(g(X))$$

$$\text{EPE}(f) = E_{X,C}(L(C, f(X)))$$

$$= E_X \left[\sum_{k=1}^L L(C_k, f(X)) \Pr(C_k | X) \right]$$

Consider
sample
population
distribution

$$EPE(f) = E_{\mathcal{X}, C} (L(C, f(\mathcal{X})))$$

$$= E_{\mathcal{X}} E_{C|\mathcal{X}} [L(C, f(\mathcal{X})) | \mathcal{X}]$$

Discrete RV's Expectation

$$= E_{\mathcal{X}} \sum_{k=1}^L L[C_k, f(\mathcal{X})] Pr(C_k | \mathcal{X})$$

$$E_C(C) = \sum_{i=1}^L C_i P(C_i)$$

$$\operatorname{argmin}_f EPE(f(\mathcal{X}))$$

$\Rightarrow$ Pointwise minimization when $\mathcal{X} = x$

$$\Rightarrow \hat{f}(\mathcal{X} = x) = \operatorname{argmin}_{f(x) \in C} \sum_{k=1}^L L(C_k, f(x)) Pr(C_k | \mathcal{X} = x)$$

$$\Rightarrow \hat{f}(x) = \operatorname{argmax}_{C_k \in \left\{ \begin{matrix} C_1 \\ C_2 \\ C_3 \\ \vdots \\ C_L \end{matrix} \right\}} Pr(C_k | \mathcal{X} = x)$$



$$\begin{cases} p(C_1 | x) \\ p(C_2 | x) \\ \vdots \\ p(C_L | x) \end{cases}$$

Expected prediction error (EPE)

$$\text{EPE}(f) = E_{X, C}(L(C, f(X))) = E_X \left[\sum_{k=1}^K L(C_k, f(X)) \Pr(C_k | X) \right]$$

Consider
sample
population
distribution

- Pointwise minimization suffices

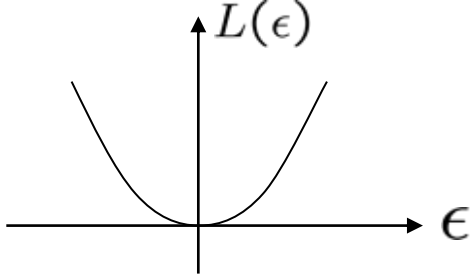
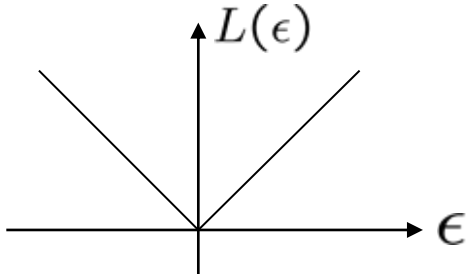
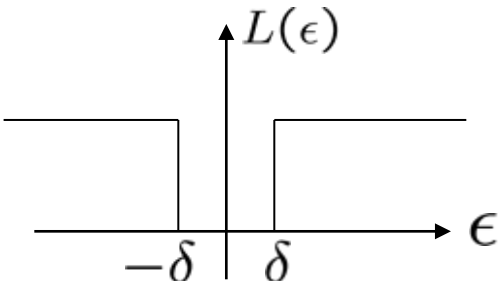
• $\rightarrow$ simply
$$\hat{f}(X) = \operatorname{argmin}_{g \in C} \sum_{k=1}^K L(C_k, g) \Pr(C_k | X = x)$$

$$\hat{f}(X) = C_k \text{ if } \Pr(C_k | X = x) \geq \Pr(C_{k'} | X = x) \text{ for all } k'$$

$$\Pr(C_k | X = x) = \max_{g \in C} \Pr(g | X = x)$$

Bayes Classifier

SUMMARY: WHEN Expected prediction error (EPE) USES DIFFERENT LOSS

Loss Function	Estimator $\hat{f}(x)$
L_2 	$EPE = E_{x,Y} (Y - f(x))^2$ $\hat{f}(x) = E[Y X = x]$
L_1 	$\hat{f}(x) = \text{median}(Y X = x)$
$0-1$ 	$\hat{f}(x) = \arg \max_Y P(Y X = x)$ (Bayes classifier / MAP)

Today: Extra

- ✓ Why Bayes Classification – MAP Rule?
 - Expected Prediction Error
 - 0-1 Loss function for Bayes Classifier

- ✓ Logistic regression

- 
- Parameter Estimation for LR

$$p(y|x) = \frac{e^{\beta x}}{1 + e^{\beta x}}$$

$\Downarrow$
 β

Logistic Regression Assumptions

- Linearity in the logit – the regression equation should have a linear relationship with the logit form of the target variable
- There is no assumption about the feature variables / target predictors being linearly related to each other.



$$\underline{P(y=1|x)} \quad 1-p(y=1|x)$$

func of x
with parameter β to learn from training data

Newton's method for optimization

- The most basic **second-order** optimization algorithm
- Updating parameter with

$$\text{GD: } \theta_{k+1} = \theta_k - \alpha g_k$$

$$\text{Newton: } \theta_{k+1} = \theta_k - \underbrace{\mathbf{H}_K^{-1}}_{p \times p} \underbrace{\mathbf{g}_k}_{p \times 1}$$

Review: Hessian Matrix / n==2 case

Singlevariate → multivariate

- 1st derivative to gradient,
- 2nd derivative to Hessian

$$f(x, y)$$

$$g = \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix}$$

$$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix}$$

Review: Hessian Matrix

Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a function that takes a vector in $\mathbb{R}^n$ and returns a real number. Then the **Hessian** matrix with respect to x , written $\nabla_x^2 f(x)$ or simply as H is the $n \times n$ matrix of partial derivatives,

$$\nabla_x^2 f(x) \in \mathbb{R}^{n \times n} = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(x)}{\partial x_2 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_2^2} & \cdots & \frac{\partial^2 f(x)}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \frac{\partial^2 f(x)}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{bmatrix}.$$

Newton's method for optimization

- Making a quadratic/second-order Taylor series approximation

$$\hat{f}_{quad}(\boldsymbol{\theta}) = f(\boldsymbol{\theta}_k) + \mathbf{g}_k^T (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)^T \mathbf{H}_k (\boldsymbol{\theta} - \boldsymbol{\theta}_k)$$

Finding the minimum solution of the above right quadratic approximation (quadratic function minimization is easy !)

$$\hat{f}(\theta) = f(\theta_k) + g_k^T (\theta - \theta_k) + \underbrace{\frac{1}{2} (\theta - \theta_k)^T H_k (\theta - \theta_k)}_{\Downarrow} \\ \frac{1}{2} (\theta^T H_k \theta - 2 \theta^T H_k \theta_k + \theta_k^T H_k \theta_k)$$

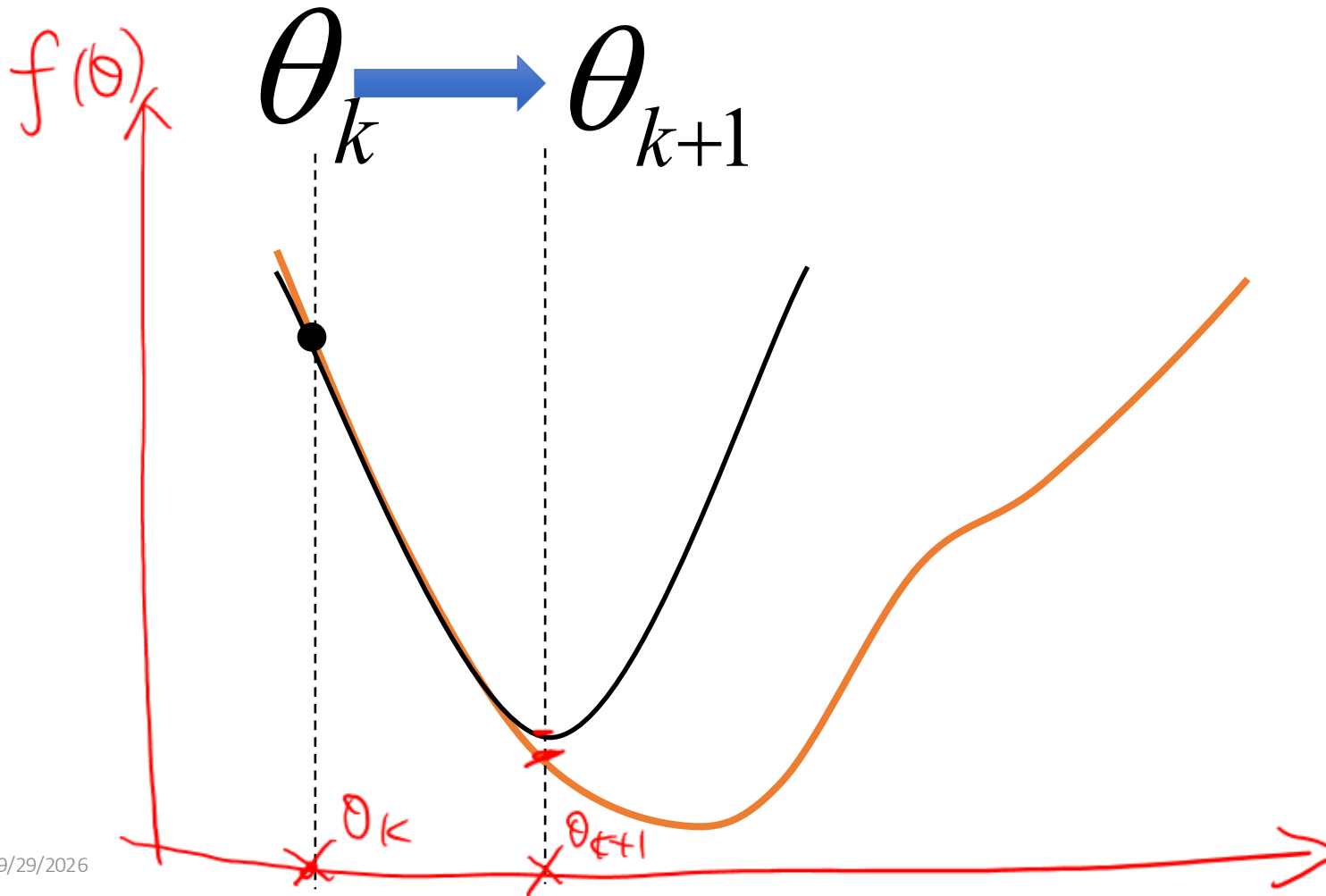
$$\frac{\partial \hat{f}(\theta)}{\partial \theta} = 0 + g_k + \underbrace{\frac{2}{2} H_k \theta - \frac{2}{2} H_k \theta_k}_{\text{see p24 handout}} = 0$$

$$g_k + H_k (\theta - \theta_k) = 0$$

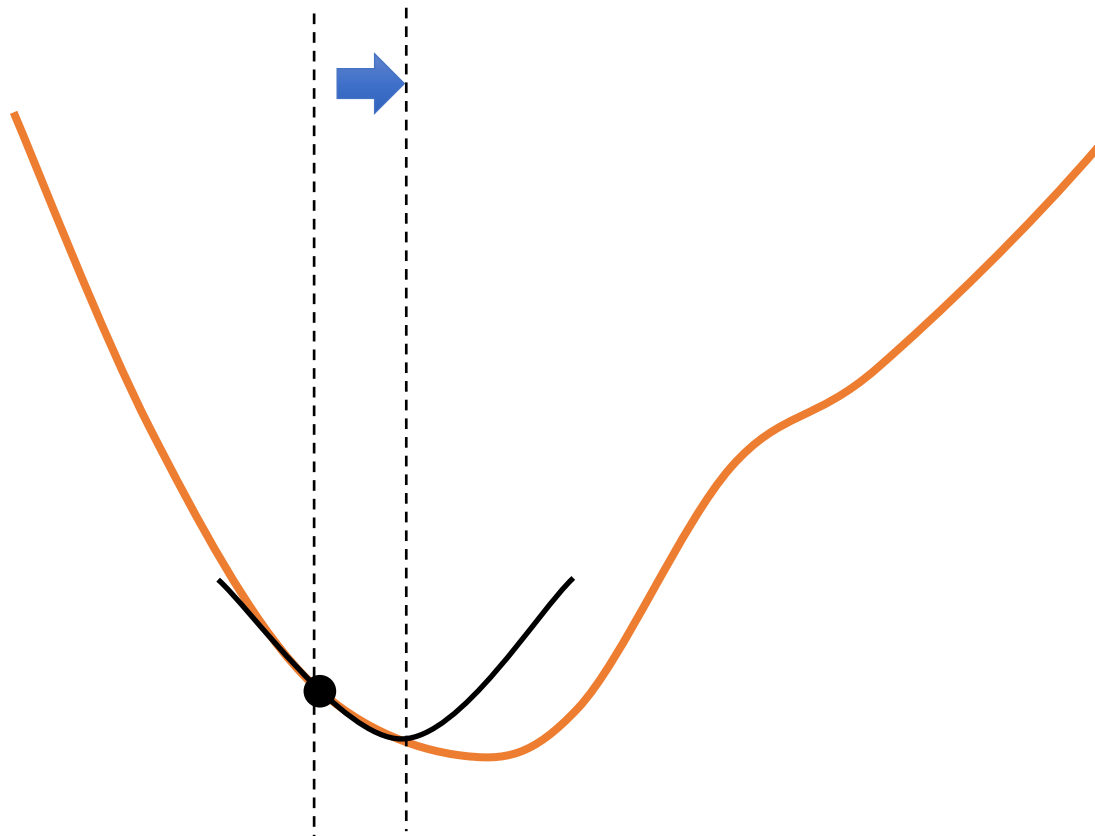
$$\Rightarrow \theta = \theta_k - H_k^{-1} g_k$$

where $H_k \in \mathbb{R}^{p \times p}$
 $g_k \in \mathbb{R}^p$

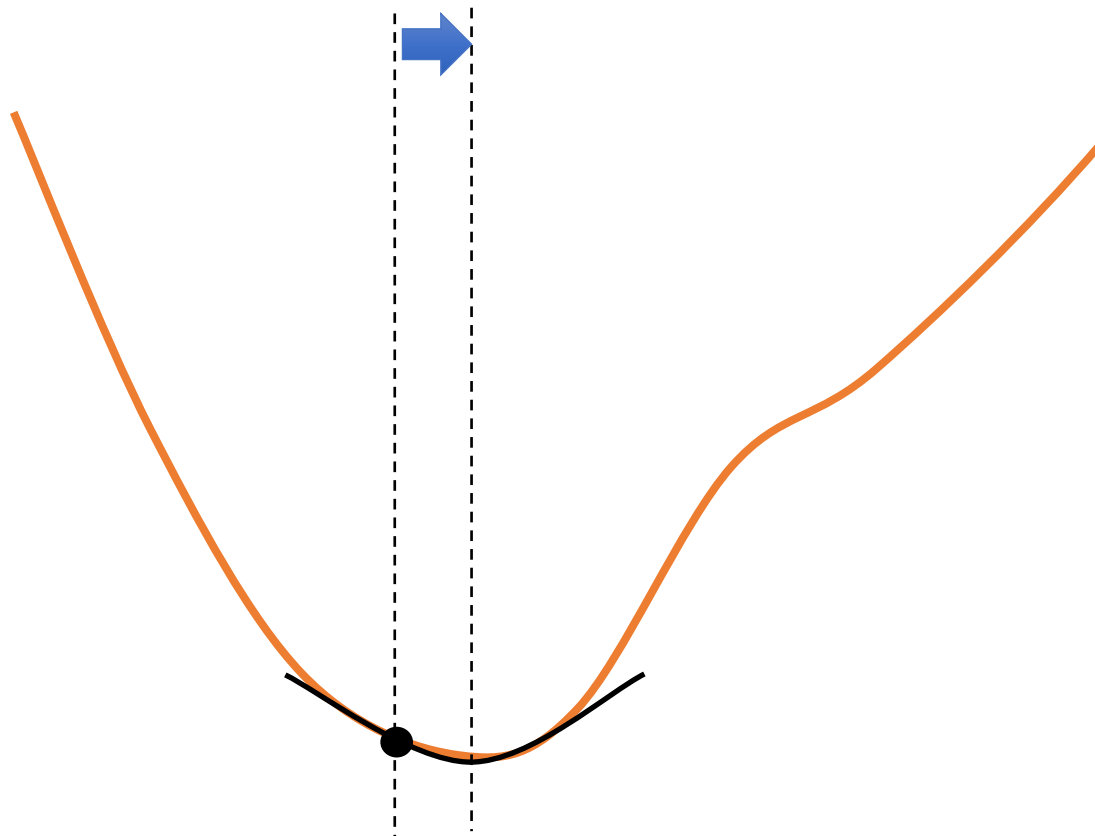
Newton's Method / second-order Taylor series approximation



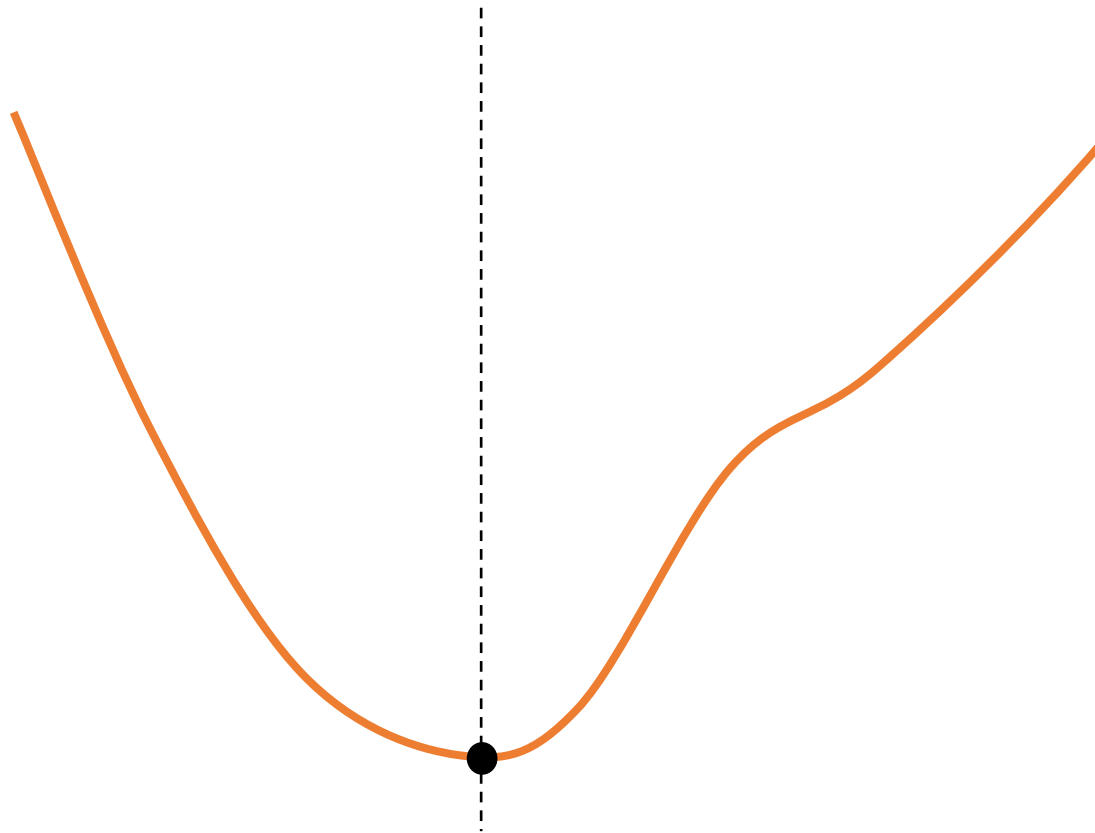
Newton's Method / second-order Taylor series approximation



Newton's Method / second-order Taylor series approximation



Newton's Method / second-order Taylor series approximation



Newton's Method

- At each step:

$$\theta_{k+1} = \theta_k - \frac{f'(\theta_k)}{f''(\theta_k)}$$

$$\theta_{k+1} = \theta_k - H^{-1}(\theta_k) \nabla f(\theta_k)$$

- Requires 1st and 2nd derivatives
- Quadratic convergence
- ➔ However, finding the inverse of the Hessian matrix is often expensive

Newton vs. GD for optimization

- **Newton:** a quadratic/second-order Taylor series approximation

$$\hat{f}_{quad}(\boldsymbol{\theta}) = f(\boldsymbol{\theta}_k) + \mathbf{g}_k^T (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)^T \mathbf{H}_k (\boldsymbol{\theta} - \boldsymbol{\theta}_k)$$

$\Rightarrow \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{1}{H(\boldsymbol{\theta}_k)} g(\boldsymbol{\theta}_k)$

Finding the minimum solution of the above right quadratic approximation (quadratic function minimization is easy !)

$$\hat{f}_{quad}(\boldsymbol{\theta}) = f(\boldsymbol{\theta}_k) + \mathbf{g}_k^T (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)^T \frac{1}{\alpha} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)$$

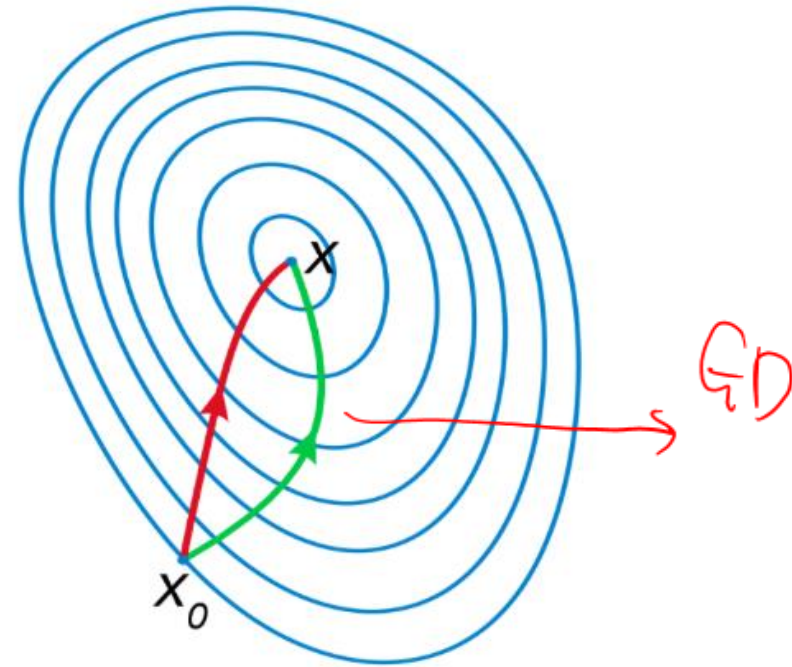
$\Downarrow \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha g(\boldsymbol{\theta}_k)$

Comparison

- Newton's method vs. Gradient descent

A comparison of gradient descent (green) and Newton's method (red) for minimizing a function (with small step sizes).

Newton's method uses curvature information to get a more direct route ...



MLE for Logistic Regression Training

Let's fit the logistic regression model for $K=2$, i.e., number of classes is 2

Training set: $(x_i, y_i), i=1, \dots, N$

For Bernoulli distribution

$$p(y | x)^y (1 - p)^{1-y}$$

(conditional)
Log-likelihood:

How?

$$\begin{aligned} l(\beta) &= \sum_{i=1}^N \{\log \Pr(Y = y_i | X = x_i)\} \\ &= \sum_{i=1}^N y_i \log(\Pr(Y = 1 | X = x_i)) + (1 - y_i) \log(\Pr(Y = 0 | X = x_i)) \\ &= \sum_{i=1}^N \left(y_i \log \frac{\exp(\beta^T x_i)}{1 + \exp(\beta^T x_i)} + (1 - y_i) \log \frac{1}{1 + \exp(\beta^T x_i)} \right) \\ &= \sum_{i=1}^N (y_i \beta^T x_i - \log(1 + \exp(\beta^T x_i))) \end{aligned}$$

x_i are $(p+1)$ -dimensional input vector with leading entry 1
 β is a $(p+1)$ -dimensional vector

$$l(\beta) = \sum_{i=1}^N \{\log \Pr(Y = y_i | X = x_i)\}$$

y_i

$p(y_i=1|x)$

$$\begin{aligned} & \log \{ \Pr(Y = y_i | X = x_i) = p(y_i | x_i) \} \Rightarrow \begin{matrix} y_i = 1 \\ y_i = 0 \end{matrix} \\ & = \log \{ p(y_i=1|x)^{y_i} (1 - p(y_i=1|x))^{1-y_i} \} \\ & = y_i \log p(y_i=1|x) + (1-y_i) \log (1 - p(y_i=1|x)) \end{aligned}$$

Newton-Raphson for LR (optional)

$$\frac{\partial l(\beta)}{\partial \beta} = \sum_{i=1}^N (y_i - \frac{\exp(\beta^T x)}{1 + \exp(\beta^T x)}) x_i = 0$$

(p+1) Non-linear equations to solve for (p+1) unknowns

Vector β

Solve by Newton-Raphson method:

$$\beta^{new} \approx \beta^{old} - \left[\frac{\partial^2 l(\beta)}{\partial \beta \partial \beta^T} \right]^{-1} \frac{\partial l(\beta)}{\partial \beta},$$

$$\text{where, } \left(\frac{\partial^2 l(\beta)}{\partial \beta \partial \beta^T} \right) = - \sum_{i=1}^N x_i x_i^T \left(\frac{\exp(\beta^T x_i)}{1 + \exp(\beta^T x_i)} \right) \left(\frac{1}{1 + \exp(\beta^T x_i)} \right)$$

minimizes a quadratic approximation to the function we are really interested in.

$$\theta_{k+1} = \theta_k - \mathbf{H}_K^{-1} \mathbf{g}_k$$

$p(x_i; \beta)$

$1 - p(x_i; \beta)$

Newton-Raphson for LR...

$$\frac{\partial l(\beta)}{\partial \beta} = \sum_{i=1}^N \left(y_i - \frac{\exp(\beta^T x)}{1 + \exp(\beta^T x)} \right) x_i = X^T (y - p)$$

$\rightarrow p(y=1|x) = \frac{e^{\beta^T x}}{1 + e^{\beta^T x}}$

$$\left(\frac{\partial^2 l(\beta)}{\partial \beta \partial \beta^T} \right) = -X^T W X$$

So, NR rule becomes:

$$\beta^{new} \leftarrow \beta^{old} + (X^T W X)^{-1} X^T (y - p),$$

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix}_{N \times (p+1)}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}_{N \times 1}, \quad p = \begin{bmatrix} \exp(\beta^T x_1) / (1 + \exp(\beta^T x_1)) \\ \exp(\beta^T x_2) / (1 + \exp(\beta^T x_2)) \\ \vdots \\ \exp(\beta^T x_N) / (1 + \exp(\beta^T x_N)) \end{bmatrix}_{N \times 1},$$

$X: N \times (p+1)$ matrix of x_i

$y: N \times 1$ matrix of y_i

$p: N \times 1$ matrix of $p(x_i; \beta^{old})$

$W: N \times N$ diagonal matrix of $p(x_i; \beta^{old})(1 - p(x_i; \beta^{old}))$

$$\left(\frac{\exp(\beta^T x_i)}{(1 + \exp(\beta^T x_i))} \right) \left(1 - \frac{1}{(1 + \exp(\beta^T x_i))} \right)$$

Newton-Raphson for LR...

- Newton-Raphson

$$\begin{aligned}
 - \beta^{new} &= \beta^{old} + (X^T W X)^{-1} X^T (y - p) \\
 &= (X^T W X)^{-1} X^T W (X \beta^{old} + W^{-1} (y - p)) \\
 &= (X^T W X)^{-1} X^T W z
 \end{aligned}$$

Re expressing
Newton step as
weighted least
square step

- Adjusted response

$$z = X \beta^{old} + W^{-1} (y - p)$$

- Iteratively reweighted least squares (IRLS)

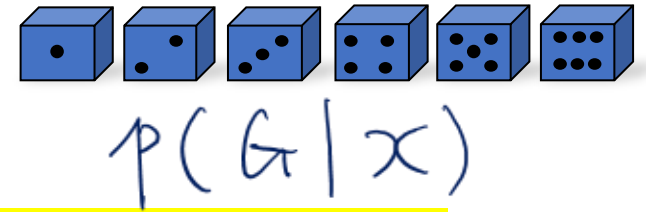
$$\beta^{new} \leftarrow \arg \min_{\beta} (z - X \beta^T)^T W (z - X \beta^T)$$

$$\leftarrow \arg \min_{\beta} (y - p)^T W^{-1} (y - p)$$

Handwritten notes in red ink:

- A red arrow points from the term $(X^T W X)^{-1} X^T W z$ in the equation above to the handwritten expression $(X^T W X)^{-1} X^T W z$.
- The handwritten expression $(X^T W X)^{-1} X^T W z$ is written in red ink, with $X^T W X$ and $X^T W z$ underlined.

Binary → Multinoulli Logistic Regression Model



Directly models the posterior probabilities as the output of regression

$$\Pr(G = k | X = x) = \frac{\exp(\beta_{k0} + \beta_k^T x)}{1 + \sum_{l=1}^{K-1} \exp(\beta_{l0} + \beta_l^T x)}, \quad k = 1, \dots, K-1$$

$$\Pr(G = K | X = x) = \frac{1}{1 + \sum_{l=1}^{K-1} \exp(\beta_{l0} + \beta_l^T x)}$$

x is p -dimensional input vector

β_k^T is a p -dimensional vector for each class k

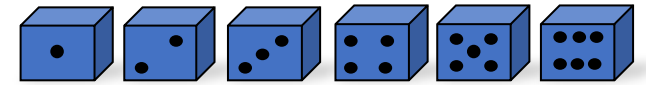
Total number of parameters is $(K-1)(p+1)$

$\beta_{k0}, \vec{\beta}_k, k=1, 2, \dots, K-1$

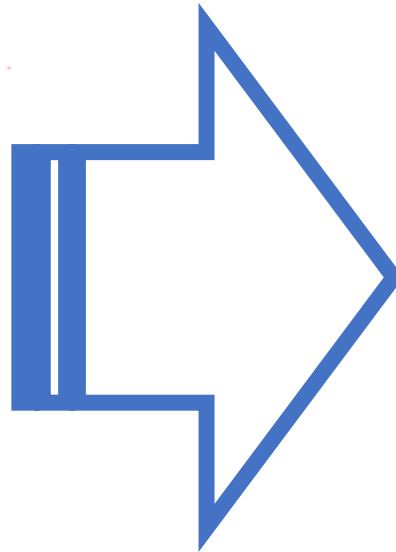
Note that the class boundaries are linear

Binary $\rightarrow$ Multinoulli Logistic Regression Model

(e.g. $k=6$)



$$p(y=1|x) = \frac{e^{\beta_1 x}}{1 + e^{\beta_1 x}}$$
$$p(y=0|x) = \frac{1}{1 + e^{\beta_1 x}}$$



$$\frac{e^{\beta_k^T x}}{1 + e^{\beta_1^T x} + e^{\beta_2^T x} + \dots}$$

e.g.

$$\ln \frac{P(G=k|x)}{P(G=k'|x)} = 0 \Rightarrow \text{linear}$$
$$\beta_{k0} + \beta_k^T x$$

Note that the class boundaries are **linear**

References

- Prof. Tan, Steinbach, Kumar's "Introduction to Data Mining" slide
- Prof. Andrew Moore's slides
- Prof. Eric Xing's slides
- Prof. Ke Chen NB slides
- Hastie, Trevor, et al. *The elements of statistical learning*. Vol. 2. No. 1. New York: Springer, 2009.